INDEPENDENT PRODUCT MANAGEMENT CASE STUDY

OWNLY — Product Problem Discovery & Solution Case Study

From Customer Signals to Product Decision

PRODUCT Ownly (food delivery, Rapido ecosystem)

DOMAIN Food Delivery / Consumer Marketplace

PREPARED BY Mahesh Kontham

DATE 8 October 2026

EVIDENCE BASE 1,481 Play Store reviews across 3 Ownly apps · Semrush organic data · press (secondary)

This is an independent analysis built only on public signals. It contains no Ownly internal data, and nothing in it has been launched or tested. Every number is labelled by how much it can be trusted.

Contents

02 Executive summary 3
03 Product & business context 4
04 Problem discovery methodology 6
05 Evidence repository 7
06 Customer review analysis 9
07 Signal → pattern analysis 12
08 Competing problem hypotheses 14
09 Root cause analysis 15
10 Customer journey & product funnel 17
11 Problem definition 18
12 Opportunity sizing 19
13 Business → product outcome tree 21
14 Competitive analysis 22
15 Solution space 23
16 Solution prioritization 24
17 MVP definition 26
18 Product experience & wireframes 28
19 PRD — Ownly Promise v1 29
20 Experiment design 31
21 Metrics & instrumentation 33
22 Risks & second-order effects 34
23 Launch & rollout plan 35
24 Expected impact & decision framework 36
25 Future iterations 37
26 Key learnings 38
27 Appendix 39

How to read the evidence labels

Label	Meaning in this document
ACTUAL / FACT	Counted directly from the collected data, or stated in a cited source. Re-computable.
OBSERVATION	Something users describe or that is visible in the data, taken at face value.
INFERENCE	A conclusion drawn by combining observations. Plausible but not proven.
ASSUMPTION	A value or belief adopted so the analysis can proceed. Must be replaced with real data.
HYPOTHESIS	A testable causal claim with a defined way to prove it wrong.
ESTIMATE	A number derived from assumptions with the formula shown.
EXPECTED	A result we expect from a test that has not run. Not an outcome.
UNKNOWN / REQUIRES VALIDATION	We do not have the data. Stated plainly instead of guessed.
SECONDARY	Press or prior notes, recorded in the source pack but not re-verified for this document.

02 Executive summary

31.6%	1.05★	43%	0
of 1–2★ customer reviews describe paying and getting no food ACTUAL	average rating when that happens ACTUAL	of reasoned 4–5★ reviews praise price / no fees ACTUAL	internal order metrics available to size it UNKNOWN

Context. Ownly is Rapido's food-delivery app. It is reported to charge restaurants no commission and customers no platform fee, and it is prepaid only SECONDARY. It runs three apps: customer, restaurant partner, and captain (rider).

Problem discovered. Across 1,000 customer reviews (27 Sep–4 Oct 2026), the worst-rated experience is not slow food or a bad menu. It is **paying and getting nothing**: the order is cancelled after payment, no captain arrives, or the order is marked "Delivered" without a handover. 156 of 493 negative reviews (31.6%) describe one of these. When they do, the rating is ~1★ whether or not support is also mentioned ACTUAL.

Why it matters. Ownly's advantage is price, and that is working: even 42 angry reviewers still praise it. But a prepaid, low-margin model cannot absorb a failed order. Every failure costs the full order value plus support time, and generates public "fraud" language (74 negative reviews use fraud/scam wording) ACTUAL. Since July 2026 Rapido's app sends new users to Ownly SECONDARY, so a failure is also a failure of Rapido's distribution.

Triangulation. The restaurant and captain apps describe the same breakdowns from the other side: order alerts that don't ring so orders auto-cancel, captains sent too early, and opaque captain incentive rules. All three apps share one pattern: a manual queue with no visible status and no owner INFERENCE.

Root-cause hypothesis. No step in the order pipeline has to *prove* a handoff happened, and no rule makes the customer whole automatically when one fails HYPOTHESIS.

Business impact. Cannot be sized from public data. The severe-failure rate per order and the repeat-order gap after a failure are both UNKNOWN. The sensitivity model (Section 12) shows the direct financial return is small at Ownly's thin margin, so the case rests on retention, trust and Rapido brand protection. That makes **sizing the failure rate the first deliverable**, before building anything.

Recommendation and MVP. Run a 2-week sizing gate. If severe failures are at least 1% of orders, or the 30-day repeat gap is 10 percentage points or more, build **Ownly Promise v1**: a 4-digit delivery code the captain must enter to mark an order delivered, plus three automatic refund rules (cancelled after payment, no captain within 20 minutes, "not received" on an unverified delivery). Every failure is logged with a cause code.

Validation and success metric. A zone-randomised pilot in one city (Bengaluru, subject to data) for at least 4 weeks. **Primary KPI:** "not received" claims per 1,000 delivered orders. **North Star:** weekly *promise-kept orders*. **Guardrails:** captain food-order acceptance, handover time, refund cost and abuse rate. Scale on a ≥40% drop with healthy guardrails; reject at <15%.

03 Product & business context

What Ownly is

Topic	What we know	Status
Ownership	Food-delivery product of Rapido (ride-hailing). Piloted in Bengaluru Aug 2025, city-wide launch 3 Mar 2026	SECONDARY press, via source pack
Distribution	Integrated into the main Rapido app in Jul 2026, still available as a standalone app	SECONDARY
Restaurant offer	Zero commission; ~20,000 restaurants listed	SECONDARY ; one merchant review mentions a “fair commission structure”, so the exact terms are UNKNOWN
Customer offer	Menu prices match the restaurant's own; no platform fee; delivery fee ~₹30 + GST	SECONDARY ; 2 reviews dispute price parity
Payment	Prepaid only (no cash on delivery)	INFERENCE from 12 reviews asking for COD
Scale	~5,000 orders/day in Bengaluru (Mar 2026). Current volume unknown	SECONDARY (stale) / UNKNOWN
Cities	Bengaluru and Hyderabad appear in reviews; Delhi, Mumbai, Chennai and Lucknow appear only in search keywords	INFERENCE
Three apps	Customer app; restaurant partner app; **Ownly Captain** rider app (riders compare it with the Rapido Captain app)	ACTUAL from scraped Play Store data

Marketplace structure

Ownly coordinates three sides it does not fully control. A customer pays upfront; the restaurant must accept and prepare the order; a captain must collect and hand it over. Ownly's operations sit in the middle and handle support, refunds, onboarding and dispatch.

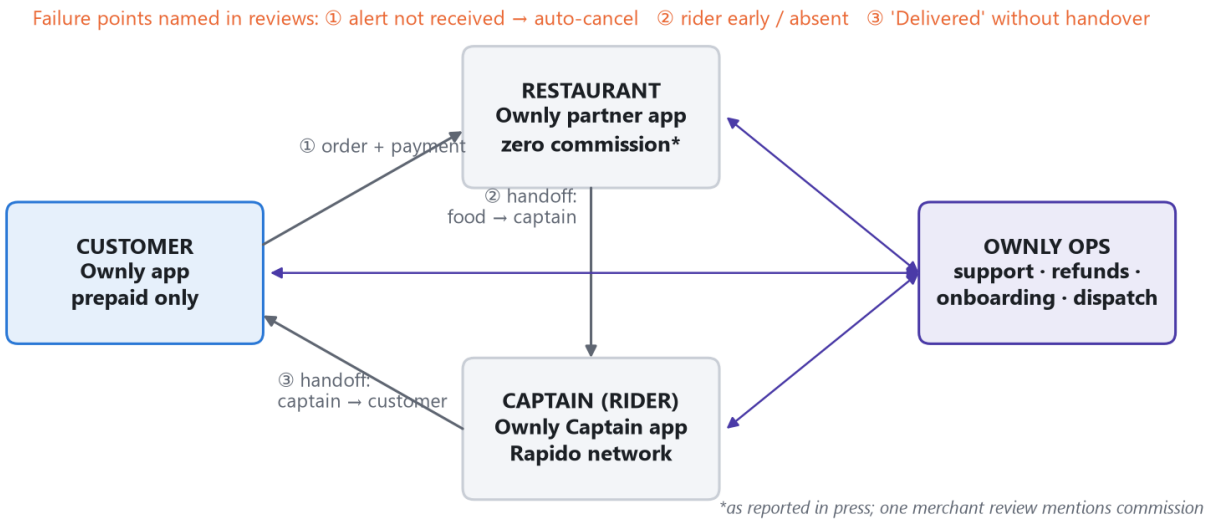


Exhibit 1. Ownly's three-sided marketplace and the three handoffs named in reviews.

Economics that matter for this problem

Incumbents (Swiggy, Zomato) are reported to earn 15–30% commission plus a ₹15–18 platform fee SECONDARY. Ownly gives most of that up and earns mainly the delivery fee. **Inference:** Ownly's margin per order is thin and roughly fixed, so a failed order (refund + support + captain time) can wipe out the margin of many good orders. A commission model

has more room to absorb failures. This is why reliability is more important to Ownly's model than to the incumbents' INFERENCE.

> IMPORTANT ASSUMPTION · **The economics are not verified**
> Contribution per order, delivery cost per trip and refund cost are not public. This document uses ₹5 / ₹10 / ₹15 contribution scenarios and a ~₹30 fee only to show how sensitive the conclusion is. Replace them with finance data before any build decision.

Competitive context in one line

Ownly wins on price and, on the evidence, loses on reliability and support. Customers say so directly: *"Zomato, Swiggy is way more good than this"* (1★, 2 Oct 2026). One reviewer praises the pricing *"in the age of institutionalised fleecing by Swiggy and Zomato"* and in the same review calls customer service useless (1★, 4 Oct 2026) OBSERVATION.

Why this problem could matter strategically

- **Price is the moat, reliability is the leak.** Incumbents can improve reliability more easily than they can drop commission, which would cut into their own profit. Ownly needs to be reliable enough, not best in class INFERENCE.
- **Rapido's distribution makes first orders the moment of truth.** Rapido users arrive at Ownly through its super-app. If their first order fails, that distribution is wasted. First-order reviews average 2.05★ (n=22) ACTUAL, a small sample.
- **Brand spillover.** 14 customer reviews name Rapido, 10 of them negative, e.g. *"side gig in between rapido rides"* ACTUAL. A food failure is also a Rapido brand risk.

04 Problem discovery methodology

The problem was not chosen first and then supported. Each source was read for what it can and cannot prove, and only patterns that appeared in **more than one independent source** were promoted to a problem hypothesis.

Source	What was analysed	What it can tell us	Limitations
Customer app reviews (Play Store)	1,000 reviews, 27 Sep–4 Oct 2026; 87% on v2.0.4; keyword-coded into 17 themes, then re-checked	What goes wrong, how severely, and in which journey stage	8 days only; Android only; self-selected (extremes over-represented); keyword tagging has false positives; cannot show frequency per order
Restaurant partner app reviews	105 reviews, 14 Nov 2025–4 Oct 2026; 33 describe a problem; analyst-coded themes	Supply-side causes of customer failures; onboarding and support friction	Small sample; 37 reviews posted on 2 days; 0 developer replies
Captain (rider) app reviews	376 reviews, 5 Mar–4 Oct 2026; keyword-coded into 7 themes plus rider stage	Captain supply, onboarding, incentives, dispatch	Most reviewers are applicants, not active captains; bursts of short 5★ reviews
Semrush organic data (ownly.food)	Overview + positions PDFs, India desktop, 5 Oct 2026; Compare Domains screenshot	Brand discovery, search demand, relative scale vs incumbents	Modelled estimates; desktop only in a mobile-first market; the “Top Opportunities” panel uses the US database
Press (secondary)	Business Today (28 Jul 2026), Upstox, as recorded in the source pack	Business model, launch dates, reported scale	Company-sourced claims; order volume is from Mar 2026; not re-verified here
Prior analysis (source pack)	Earlier workbook analyses and strategy notes	Starting hypotheses	Re-tested here; four claims corrected (Appendix C)

Why multiple sources were necessary

- **Reviews alone over-sample anger.** A customer whose order worked rarely writes 40 words; a customer who lost ₹430 does. The restaurant and captain apps test whether the customer story has a real operational mechanism behind it.
- **Different sides see different halves of one failure.** A customer sees “cancelled after payment”. A restaurant sees “the ring didn't fire, the order auto-cancelled, and I was blamed”. Only together do they point to a cause.
- **Search data tests the growth alternative.** If discovery were the constraint, it would show up as a search gap. It does, but the data shows distribution runs through Rapido, not search (Section 8).

Method in four steps

1. **Code** every review into themes (keyword dictionary for customer and captain apps; analyst coding for the restaurant app) and map themes to journey stages.
2. **Measure** each theme on frequency (share of 1–2★ reviews), severity (average rating when mentioned), agreement (thumbs-up), recency and segment.
3. **Stress-test** the coding: check false positives, re-cut the strongest claims, and drop any claim that does not survive (Appendix C).
4. **Triangulate**: promote a pattern only if an independent source shows the same mechanism, then write competing hypotheses before choosing one.

05 Evidence repository

The signals that carry the argument are below, grouped by side. Quotes are verbatim from the scraped reviews (spelling unchanged). The full theme tables are in Appendix A.

Customer app

Source · date · n	Exact observation	Interpretation	Confidence	Limitation
Customer reviews · 27 Sep–4 Oct 2026 · n=1,000	47.6% of reviews are 1★ and 44.3% are 5★; average 2.96★	Polarised by incidents rather than a mediocre product	HIGH	Self-selection exaggerates both ends
Same · n=493 (1–2★)	104 reviews say the order was marked delivered or never arrived; avg 1.07★. *"after 10 mins it showed that the order is delivered And gave me a random number"*	Handover is not verified	HIGH	Some claims may be customer error or abuse
Same	61 reviews describe cancellation after payment; avg 1.00★. *"they accepted order first and deducted the amount. After 1hour … they are asking me to cancel"*	Payment risk sits with the customer	HIGH	Cause of cancellation is not visible
Same	Refund problems in 113 reviews (avg 1.15★). *"refund stuck in initiated no money given this is 527 rs"*	Recovery is slow, partial or missing	HIGH	Refund turnaround time unknown
Same	Support unresponsive in 241 (48.9% of 1–2★). Bot complaints: *"the bot never lets you connect to a person"*	Recovery has no decision authority	MEDIUM-HIGH	Dictionary includes broad words ("mail", "chat") so this is inflated
Same · n=188 reasoned 4–5★	81 (43.1%) cite price, no platform fee or no hidden charges; 42 negative reviews still praise price	The value proposition lands	MEDIUM-HIGH	2 reviews dispute price parity
Developer replies · n=1,000	99.9% replied; median reply ~14 min; 49.8% of replies redirect to email	Fast acknowledgement, no resolution in the channel	HIGH	Reply content is templated

Restaurant partner app

Source · date · n	Exact observation	Interpretation	Confidence	Limitation
Partner reviews · 7 Aug 2026 · 4★	*"notification of order is not working properly … order getting cancelled without our knowledge. aps not working on background."*	A supply-side cause of "cancelled after payment"	MEDIUM	3–4 reviews only
Partner reviews · 17 Apr 2026 · 4★	*"riders coming to the restaurant when the order assigned … difficult to prepare in 10 mins"*	Dispatch ignores prep time	LOW-MEDIUM	2 reviews
Partner reviews · 2026	*"submit the same documents more than 10 times"*; *"called for menu updates more than 50 times"*; *"9 follow-ups"*	Partners depend on a manual ops queue	MEDIUM	Small n (33 problem reviews)
Partner reviews · 9 Jun 2026 · 1★	*"payments were reportedly sent to a bank account that does not belong to me"*	Money-movement integrity risk	LOW (n=1)	Single incident; escalate, don't generalise

Captain (rider) app

Source · date · n	Exact observation	Interpretation	Confidence	Limitation
Captain reviews · Mar–Oct 2026 · n=376	58.7% of negative reviews are about verification/activation (avg 1.34★); these hold 65% of all thumbs-up	Applicants are stuck before their first delivery	HIGH	Applicants are not yet captains
Same · n=93 stating a wait	Median stated wait 30 days; 29% over 30 days; longest 180	Activation is a black-box queue	MEDIUM	Self-reported durations
Captain review quoting Ownly social team	*"we are not onboarding Captains at the moment, so we won't be able to activate your ID"*	Supply intake is paused while sign-up stays open	MEDIUM	Single quoted message
Captain reviews · May–Aug 2026	*"not getting order 12 hrs"*; *"not getting orders in yelahanka area"*; *"Orders are coming in very less compared to the Rapido Captain app"*	Some captains are idle while customers report no captain	LOW	3 reviews; needs dispatch data
Captain reviews · Jul–Aug 2026	*"even after a rider goes offline, orders are still being assigned and counted as busy pings"*	Incentive rules feel unfair; may reduce food-order acceptance	LOW-MEDIUM	Few reviews; mechanism unverified

Search, market and rating-integrity signals

Source · date · n	Exact observation	Interpretation	Confidence	Limitation
Semrush · 5 Oct 2026	~1.1K est. monthly desktop organic visits (India); 163 keywords; #1 for "rapido food delivery bangalore"; ranks for misspellings "only app", "onely"	Discovery is borrowed from Rapido; the Ownly brand is not yet searched for	MEDIUM	Modelled; desktop only
Semrush Compare Domains	Organic traffic: ownly.food 1.1K vs zomato.com 67.9M vs swiggy.com 15.3M	Scale gap is vast, but irrelevant to the near-term problem	MEDIUM	Worldwide database
Captain + partner apps	Both apps receive review spikes on **2 Jun** and **19 Jun 2026** (partner: 19 and 18 reviews, all 5★)	Consistent with solicited review drives; headline ratings are unreliable	MEDIUM	Does not prove any review is fake

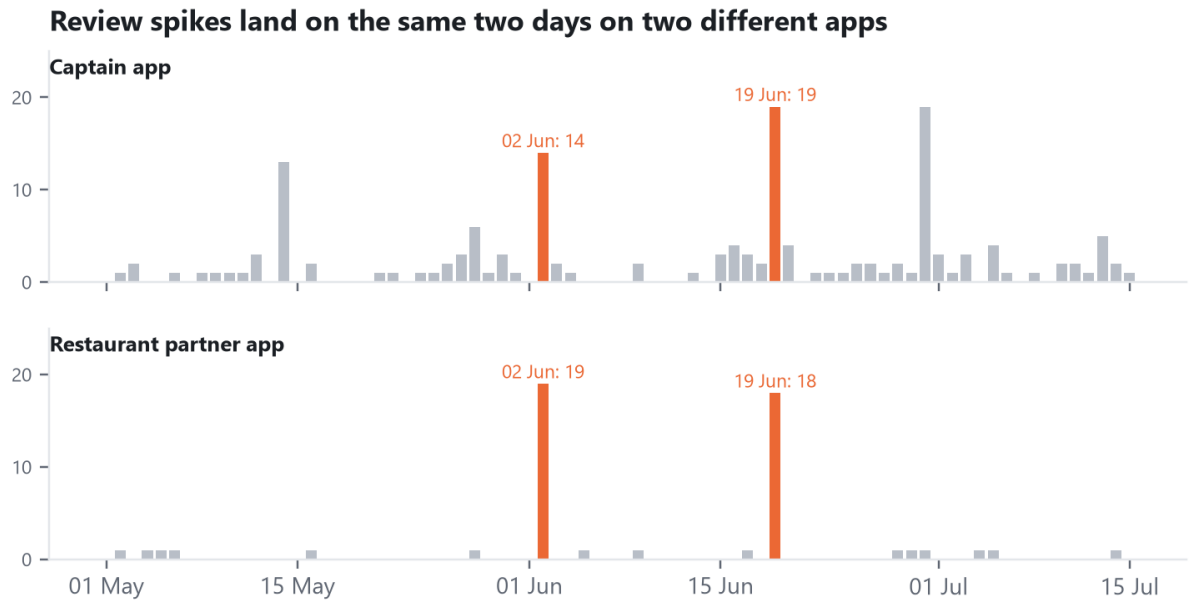


Exhibit 2. Daily review counts, May–Jul 2026. The same two dates spike on two different apps.

06 Customer review analysis

EVIDENCE · Sample and limits

1,000 customer reviews over **8 days** (27 Sep–4 Oct 2026), Android only. 493 are 1–2★. 55 negative reviews (11.2%) match no theme (e.g. “worst”, regional language). Percentages are shares of reviews, **not shares of orders**.

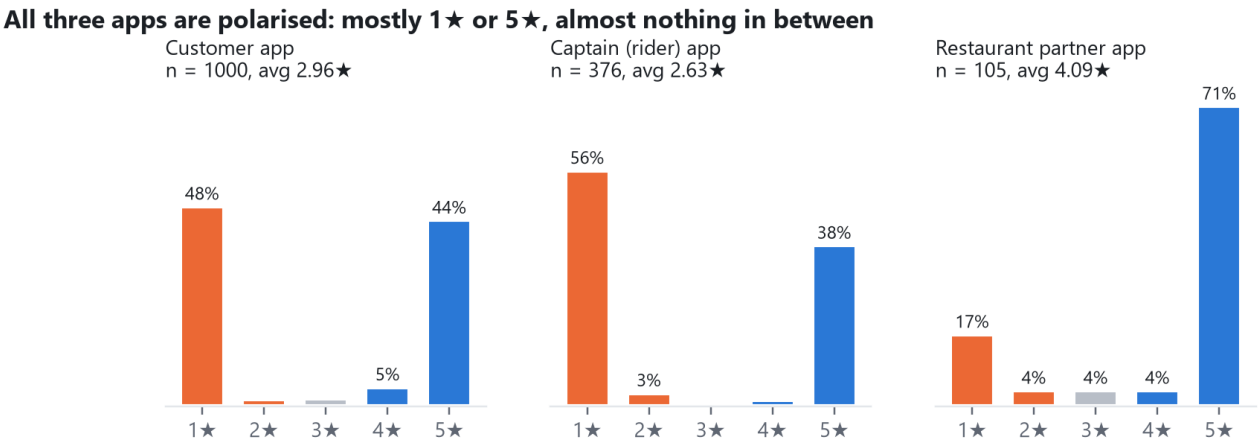


Exhibit 3. Rating distribution on all three Ownly apps.

Taxonomy

The generic taxonomy (delivery, pricing, UX, etc.) was replaced with one that follows the order journey, because the data shows failures cluster **after payment**. Themes were mapped to seven journey stages; a review can touch several.

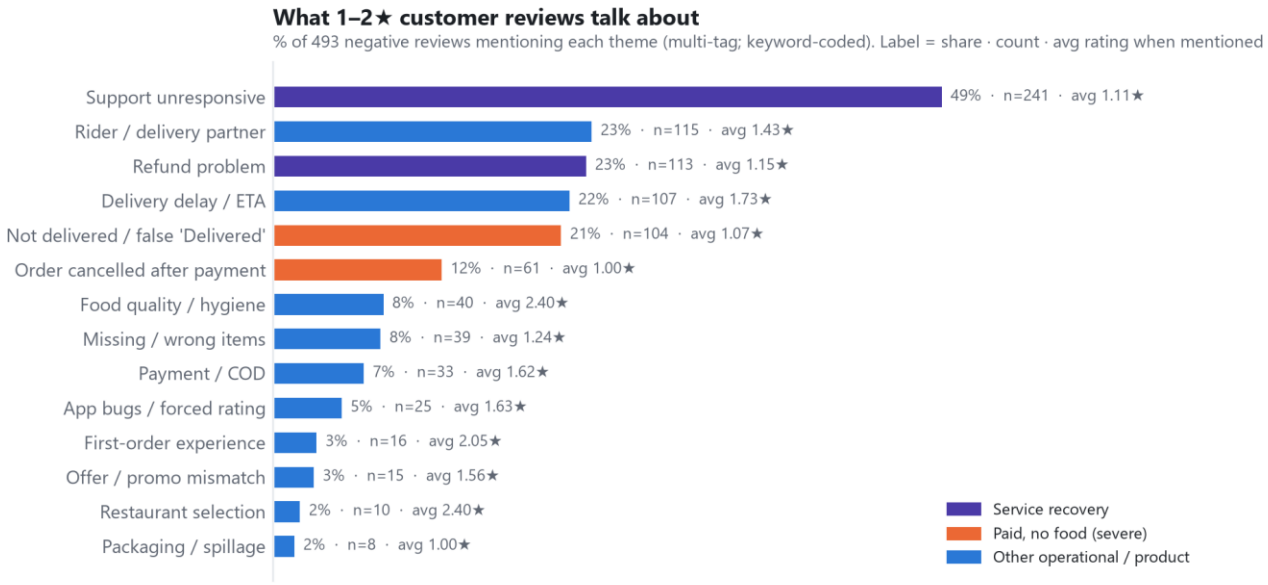


Exhibit 4. Theme frequency in 1–2★ reviews, with count and average rating when mentioned.

Frequency × severity

Frequency alone would make “support” the problem. Adding severity changes the picture: the two **paid-no-food** themes (orange) sit at the severity floor (1.00–1.07★) with meaningful frequency, while delays and food quality are milder.

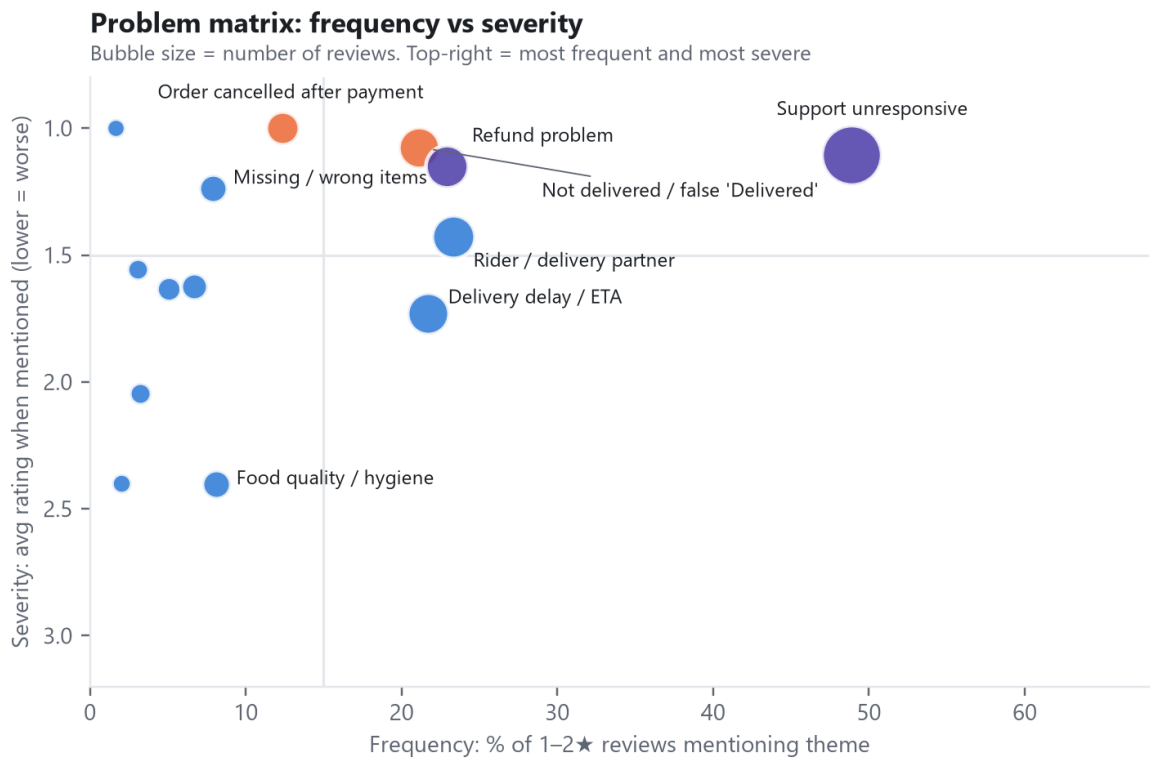


Exhibit 5. Frequency vs severity of customer review themes.

Severity, segment, recency

Dimension	Finding	Label
Severity	Union of "cancelled after payment" and "not delivered / false Delivered": 156 of 493 negatives (31.6%), avg 1.05★ across all 159 mentions	ACTUAL
Segment	First-order reviews avg 2.05★ (n=22). Rapido-mentioning reviews avg 2.07★ (n=14). Too small to segment reliably	ACTUAL / small n
Recency	Daily share of 1–2★ ranged 42%–56% across the 8 days; no single-day spike explains the pattern	ACTUAL
App version	v2.0.4 = 87% of reviews (3.05★); older v2.0.1 averages 2.21★ (n=24). Fixes may have landed, or it is noise	ACTUAL / inconclusive
Time of day	12 reviews mention night or late-night timing (e.g. *"placed an order around 1 AM, but no rider was assigned"*)	INSUFFICIENT EVIDENCE

The failure cascade

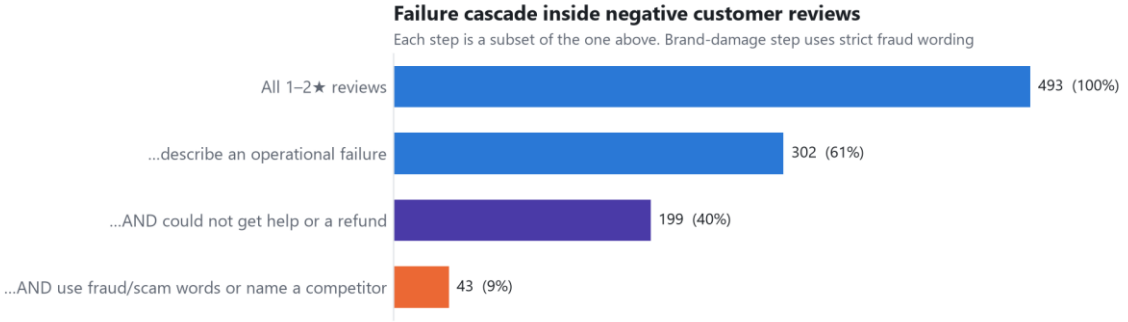


Exhibit 6. Within negative reviews: operational failure → no recovery → brand-damage language.

61% of negative reviews describe an operational failure; two-thirds of those also say they could not get help or a refund; and 43 go on to use fraud/scam words or name a competitor ACTUAL. The cascade shows the order of events in the review text. It does **not** prove that the lack of recovery caused the brand damage (see the correction below).

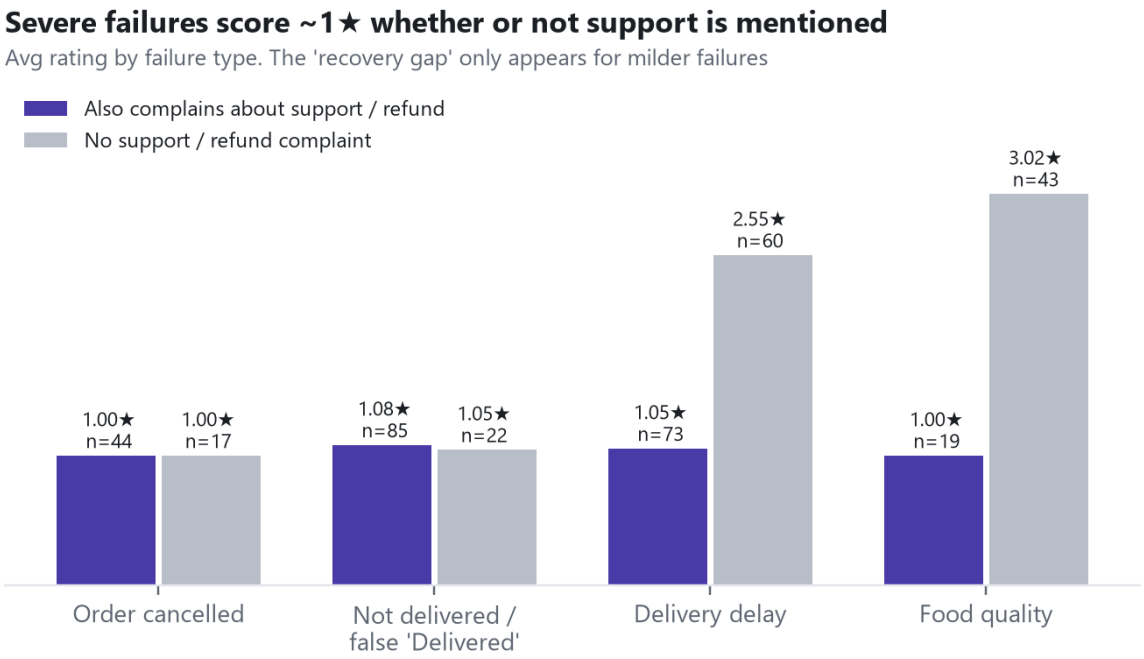


Exhibit 7. Average rating by failure type, split by whether support/refund is also criticised.

CORRECTION · **Recovery does not rescue a severe failure**

An earlier analysis concluded that "failed recovery costs a full star" (2.33★ vs 1.07★). Split by failure type, that gap disappears for severe failures: cancelled orders are 1.00★ either way, and not-delivered orders are 1.08★ vs 1.05★. The gap only exists for delays and food quality. Even there, the "no recovery complaint" group is inflated by false positives: 53 of those 159 reviews are 4–5★, and the delay dictionary includes positive phrases such as "on time" and "punctual".

Implication: prevent the severe failure first. Recovery is necessary but will not save the rating on its own.

Cross-source confirmation

Customer pattern	Restaurant app	Captain app	Status
Cancelled after payment	Alerts not ringing → auto-cancel; *"order getting cancelled without our knowledge"*	—	INFERENCE 2 sources
No rider / rider unreachable	Rider assigned at acceptance, arrives before food is ready	Captains idle (*"not getting order 12 hrs"*); onboarding paused	INFERENCE 3 sources, small n
False "Delivered"	—	Opaque incentive rules (busy-ping penalties) could encourage shortcuts	HYPOTHESIS unverified
Support doesn't resolve	*"9 follow-ups"*, *"50 times"*, 0 public replies	0 public replies; *"verification pending"* for months	INFERENCE 3 sources
Price is why people come	*"cost-saving food delivery model"* (5★)	—	FACT 2 sources

07 Signal → pattern analysis

Each pattern below is built bottom-up from signals, with facts, inferences and assumptions kept apart. Five patterns survive; two candidate patterns were dropped for lack of evidence (listed at the end).

P1 · The money-loss moment is the worst moment

Observation	Evidence	Pattern	Potential implication
Customers pay, then the order is cancelled, unassigned or falsely marked delivered	156 negative reviews; avg 1.05★; 74 negatives use fraud/scam words	Trust collapses when money disappears, not when food is late	Fix the paid-no-food path before speed, menus or discounts

FACT	INFERENCE	ASSUMPTION
Severity ~1★ for both severe themes	Prepaid-only puts all failure risk on the customer	Review prevalence roughly tracks the real failure mix

P2 · Failures happen at handoffs between parties

Observation	Evidence	Pattern	Potential implication
Restaurant → captain → customer transitions break	Customer themes + partner alerts + captain dispatch reviews	Ownly coordinates three parties but verifies none of the handoffs	The fix is orchestration and verification, not more supply

FACT	INFERENCE	ASSUMPTION
Same mechanisms are described independently on 2–3 apps	No handoff is verified today (no OTP is mentioned anywhere)	Captain-app data reflects the same fleet that serves customers

P3 · Every side is stuck in an invisible queue

Observation	Evidence	Pattern	Potential implication
Refund “initiated” for weeks; captain verification pending for a median 30 days; partner documents requested 10+ times	3 apps; developer reply rate 99.9% (customer) vs 0% (partner, captain)	Manual ops queues with no status, clock or owner	A status + SLA layer is a company-wide principle; the customer refund queue is the costliest instance

FACT	INFERENCE	ASSUMPTION
Reply rates; quoted waits	Attention goes to the side where symptoms show, not where causes start	Reply behaviour reflects internal resourcing

P4 · Price is the moat on both sides

Observation	Evidence	Pattern	Potential implication
Customers and restaurants praise the economics	43% of reasoned praise; 42 angry reviewers still praise price; partner 5★ cites cost saving	Demand is not the constraint; reliability is	Do not spend on discounts; spend on reliability

FACT	INFERENCE	ASSUMPTION
Counts above	People who leave are not leaving over price	Price parity holds for most restaurants (2 disputes)

P5 · The rating signal is distorted

Observation	Evidence	Pattern	Potential implication
Short 5★ reviews, forced prompts, same-day bursts on two apps	60.9% of customer 5★ have ≤3 words; partner 37 of 105 reviews on 2 days; captain organic avg 2.19★ vs 2.63★ headline	Store ratings cannot be used as a KPI	Measure order-level outcomes; stop review drives (Play Store policy risk)

FACT	INFERENCE	ASSUMPTION
Counts above	Bursts are solicited	—

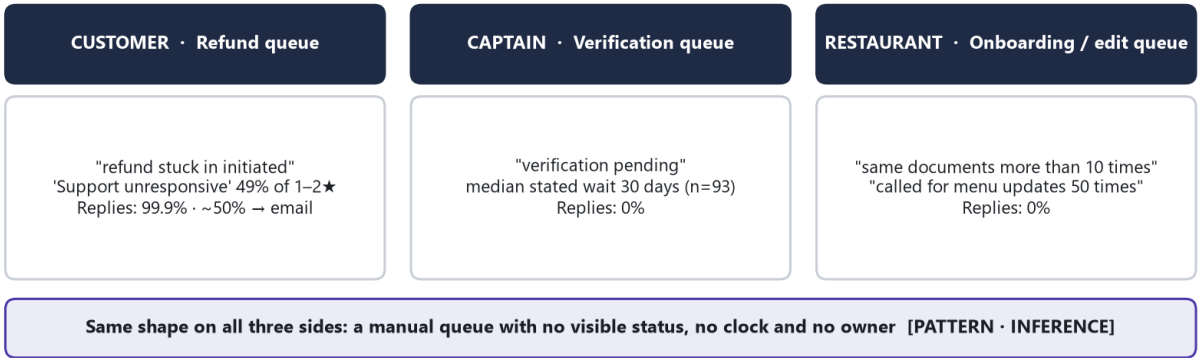


Exhibit 8. Pattern P3: the same invisible-queue shape on all three sides.

Candidate patterns dropped

- **“Night-time supply gap.”** Only 12 reviews mention night timing. INSUFFICIENT EVIDENCE. Kept as a question for dispatch data.
- **“Negative reviews dominate what new users read” (94% of helpful votes).** True but based on only **32** thumbs-up across 1,000 reviews (30 on negatives). The base is too small to support the claim, so it was removed from the argument.

08 Competing problem hypotheses

Before settling on reliability, every explanation supported by at least some evidence was written down with the evidence against it. Irrelevant hypotheses (e.g. AOV, which no source speaks to) were not forced in.

#	Hypothesis	Supporting evidence	Contradicting evidence	Confidence	Validation needed
H1	**Delivery reliability:** paid orders end with no food, and this drives churn	156 negatives at ~1★; partner alert and dispatch reviews; fraud language follows	Frequency per order unknown; reviews over-sample failures; some claims may be abuse	MEDIUM-HIGH existence · UNKNOWN size	Order logs: severe-failure rate; cohort repeat gap
H2	**Service recovery** is the primary problem	49% of negatives mention support; 85 negatives cite support with no operational failure	Severe failures are ~1★ with or without support complaints (Exhibit 7); support dictionary is broad	MEDIUM (as amplifier)	Ticket data; resolution time vs repeat
H3	**Captain liquidity:** too few captains	"No rider" reviews; late-night examples	Captains report idle hours; onboarding is paused; 150 applicants waiting. Suggests allocation, not headcount	LOW-MEDIUM	Dispatch logs: time-to-assign by zone/hour; idle captain minutes
H4	**Restaurant acceptance failures** cause cancellations	Partner reviews: alerts not ringing, auto-cancel, blamed	3–4 reviews; stock-outs or restaurant choice could also explain it	MEDIUM	Cancellation cause codes; alert delivery receipts
H5	**Restaurant selection / density** limits retention	Partners report "one order a day"	Only 2% of customer negatives mention selection	LOW	Repeat rate of customers with clean orders
H6	**Pricing** is the problem	2 price-parity disputes	Price is the #1 praise driver	REJECTED	Price-parity audit (protect the moat)
H7	**Acquisition / discovery** is the constraint	1.1K organic visits vs 15–68M for incumbents	Distribution runs through the Rapido app; fixing acquisition fills a leaky bucket	LOW (for now)	Install → first-order funnel
H8	**App UX / bugs** drive dissatisfaction	Forced rating prompt; payment glitches (5–7% of negatives)	Low frequency; ratings 1.6★, milder than the severe themes	LOW	Crash and payment-failure rates
H9	**Retention is weak regardless** (clean-order customers don't return either)	None directly; partners' low volume is consistent with it	No data	UNKNOWN	**Kill test:** if true, H1 is not the binding constraint

PM DECISION · Lead with H1, design to be proven wrong by H9

H1 has the strongest triangulated evidence and the clearest mechanism. But a strong signal in reviews is not proof of business impact. The plan therefore starts with a sizing gate that tests H1 against H9 using two queries. If severe failures are rare **and** clean-order customers also fail to return, the reliability plan stops and the team pivots to the retention/selection question.

09 Root cause analysis

Business problem → user problem → behaviour → root cause

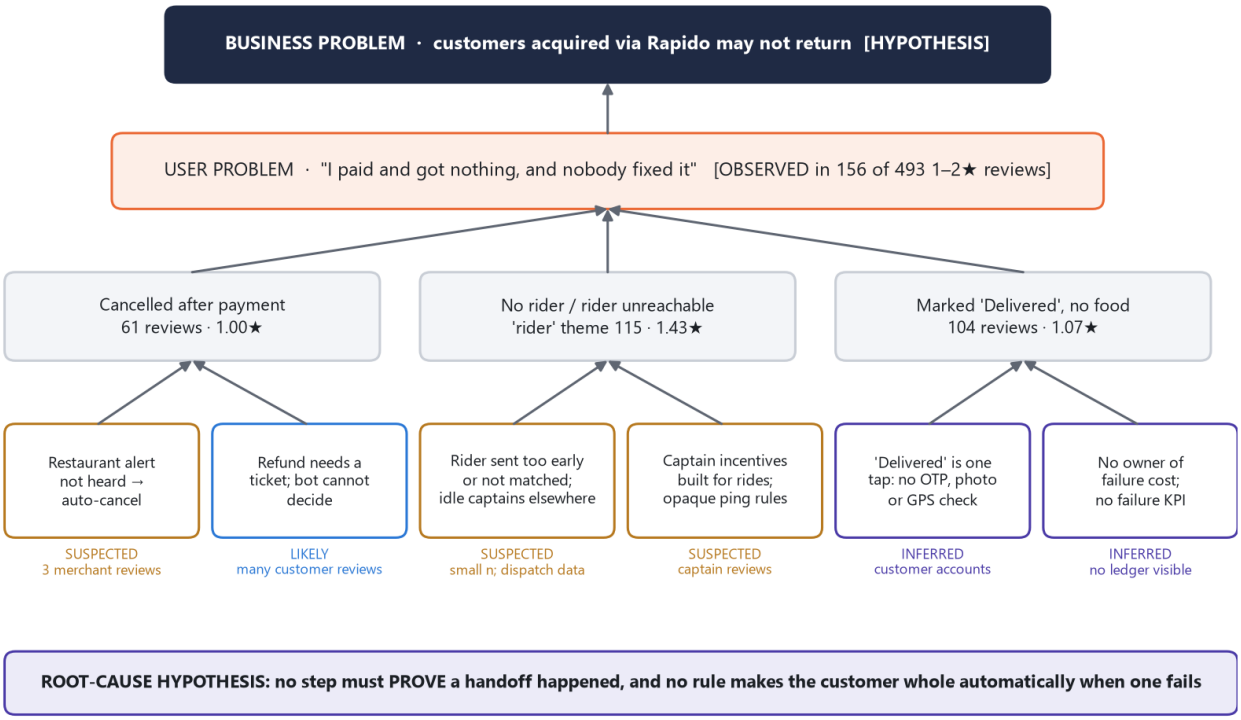


Exhibit 9. Causal tree. Border colour shows confidence: blue = likely, violet = inferred, amber = suspected.

5 Whys: "I paid, it says Delivered, I have no food"

Why?	Answer	Status
1. Why didn't the customer get food?	The order was marked delivered without a handover (wrong address, abandoned order, or bad intent)	OBSERVED in 104 reviews
2. Why could it be marked delivered?	Marking delivered appears to need one tap, with no code, photo or GPS check (*"after 10 mins it showed that the order is delivered"*)	INFERRED
3. Why wasn't it caught?	No visible anomaly detection (e.g. "delivered" far from the drop pin, or too fast)	SUSPECTED
4. Why wasn't the customer made whole?	Refunds need a ticket, the ticket goes to a bot or email, and the bot cannot decide	LIKELY
5. Why is it designed that way?	Refund liability isn't assigned to the party at fault, so the default is to delay or deny	HYPOTHESIS

Fishbone summary for "paid, no food"

Category	Contributing cause	Evidence strength
Restaurant	Order alert doesn't fire on tablets or in the background → auto-cancel; prep time not captured	SUSPECTED 3–4 partner reviews
Dispatch / captains	Captain assigned at acceptance (too early) or not at all; idle captains elsewhere; incentives tuned for rides; no thermal bags	SUSPECTED small n across 2 apps

Category	Contributing cause	Evidence strength
Product	No delivery code; status the customer cannot trust; bot with no exit; no one-tap “not received”	INFERRED consistent customer accounts
Policy / payments	Prepaid only; refunds slow, partial or stuck at “initiated”	LIKELY 113 refund reviews
Measurement	No visible failure KPI; distorted store ratings; leadership may be steering on inflated stars	INFERRED

What is confirmed vs suspected

- **Confirmed (from data):** severe failures exist, recur and score ~1★; customers describe them consistently; recovery is commonly reported as failing.
- **Likely:** refund and support flows lack decision authority.
- **Suspected, needs validation:** missed restaurant alerts as a main cause of cancellations; dispatch timing and captain allocation as a cause of no-rider failures; captain incentive design as a cause of shortcuts.

UNKNOWN · **Data required before the root cause is accepted**

- Severe-failure rate per 1,000 orders, split by cancelled-after-payment / no captain / delivered without GPS match at the drop point.
- Post-payment cancellation cause codes, and alert delivery and acknowledgement receipts per restaurant.
- Time from acceptance to captain assignment by zone and hour; idle captain minutes in the same zones.
- Refund turnaround; share of refunds that are partial, denied or stuck at “initiated” for more than 7 days.
- 30-day repeat rate: customers whose first order failed vs customers whose first order was clean.
- Whether all Ownly deliveries run through the Ownly Captain app (this changes the MVP dependency).

10 Customer journey & product funnel

No funnel data is available, so no conversion rate can be calculated UNKNOWN. What the reviews do show is **where** the journey breaks. That is overwhelmingly after the customer has paid, when they no longer control the outcome.

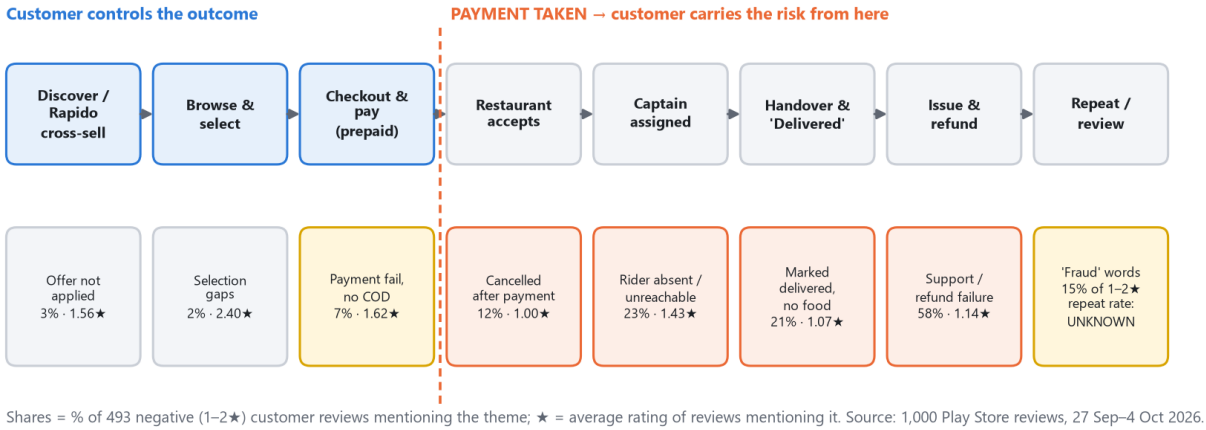


Exhibit 10. Journey map with pain points from reviews.

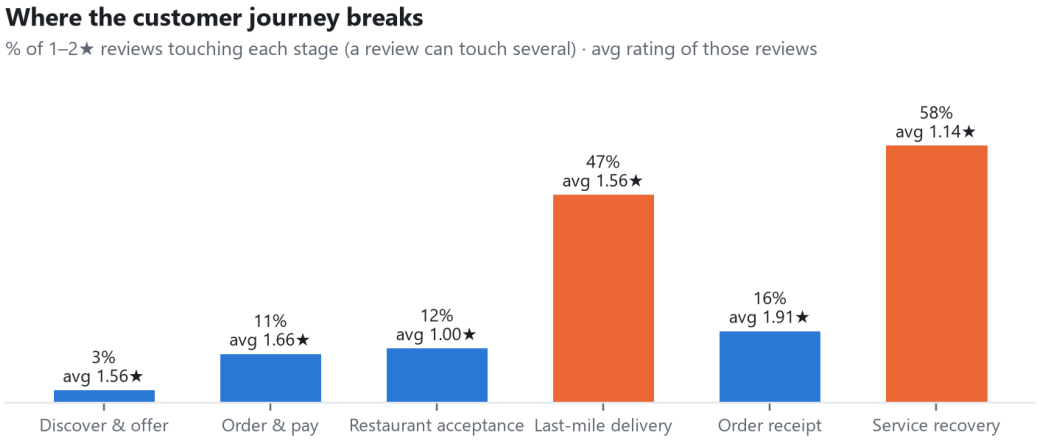


Exhibit 11. Share of negative reviews touching each stage.

Stage	Type	What reviews show	Metric we would need
Discover → install	Bottleneck?	Discovery is via Rapido; brand searched as “rapido food”	Rapido-to-Ownly entry rate UNKNOWN
First order / offer	Friction	15 negatives say the advertised offer didn't apply	Offer redemption error rate UNKNOWN
Checkout → payment	Friction / drop-off	Payment failures; money deducted with no order; COD requests	Payment success rate UNKNOWN
Restaurant accepts	**Failure point**	Cancelled after payment (61, 1.00★)	Post-payment cancellation rate by cause
Captain assigned	**Failure point**	No captain for 40–75 min; captain unreachable	Time to assign; % unassigned at 20 min
Handover	**Moment of truth**	Marked delivered, no food (104, 1.07★)	% delivered without GPS match
Issue → refund	**Moment of truth**	Bot loops, email redirects, refunds stuck	Refund turnaround; % auto-resolved
Repeat / review	Outcome	Fraud language; uninstall threats; forced rating prompt	30-day repeat rate (failed vs clean)

11 Problem definition

KEY INSIGHT · Problem statement

For prepaid Ownly customers, especially those on their first few orders, **an order can end with money taken and no food** (cancelled after payment, no captain assigned, or marked "Delivered" without a handover) **because** no step in the order pipeline has to prove the handoff happened and no rule makes the customer whole automatically when it fails, **resulting in** ~1★ reviews, "fraud" language, and (hypothesised) lost repeat orders from customers Ownly acquired through Rapido's distribution.

Question	Answer	Label
Who experiences it?	Prepaid customers in live cities; likely concentrated among first-time users arriving from the Rapido app	INFERENCE
How frequently?	Present in 31.6% of negative reviews. Per-order frequency is unknown	ACTUAL / UNKNOWN
How severe?	The most severe experience in the dataset: 1.00–1.07★ average	ACTUAL
Why now?	Rapido-app integration (Jul 2026) increases the number of first orders exposed to the weakest step	SECONDARY + INFERENCE
Why should Ownly care?	A failure costs the full order with a thin margin to absorb it, and turns a price-driven customer into a public detractor	INFERENCE
Why strategic?	It protects the one advantage incumbents can't easily copy (fair price) by removing the reason to leave	INFERENCE

Alternative problem framings considered and rejected

Framing	Why it was rejected (for now)
"Support is bad"	Support complaints mostly sit on top of a failure, and do not change the rating of severe failures. Fixing support alone adds cost linearly while the failures continue
"Delivery is slow"	Delay reviews are milder (1.73★) and the theme includes false positives ("on time"). Speed is not what triggers fraud language
"Not enough captains"	Contradicted by idle-captain and paused-onboarding signals. Reframed as allocation and dispatch, pending data
"Not enough restaurants"	Only 2% of negatives mention selection
"Ownly needs more traffic / SEO"	Distribution comes from Rapido; more traffic would increase the number of customers hitting failures
"Customers want COD"	12 requests. COD would add captain cash risk and does not stop cancellation or false delivery

12 Opportunity sizing

> **IMPORTANT ASSUMPTION · No internal data was available**
>
> Every input below is an ASSUMPTION unless labelled otherwise. The model is built per **10,000 orders/day (300,000/month)** as a scaling unit, not as a claim about Ownly's volume. Press reported ~5,000/day in Bengaluru in March 2026 SECONDARY; current volume is UNKNOWN. The purpose is to show **which inputs decide the answer**.

Inputs

Input	Conservative	Base	Upside	Label / source
Orders per month (scaling unit)	300,000	300,000	300,000	ASSUMPTION
Severe failure rate f (paid, no food)	0.75%	1.5%	3%	ASSUMPTION; replaced in week 1
Share of severe failures that are false/not delivered	40%	40%	40%	ASSUMPTION: reviews show ~65% (107 of 164 severe-failure reviews incl. no-rider); 40% used to be conservative, since reviews likely over-represent the most dramatic failure
Delivery code (OTP) cuts false deliveries by	30%	50%	70%	ASSUMPTION
Auto-refund + timeouts cut other severe failures' churn effect by	15%	25%	35%	ASSUMPTION
Average order value	₹300	₹300	₹300	ESTIMATE; reviews cite ₹135–₹527
Repeat gap: failed vs clean customer (30-day)	10 pp	20 pp	30 pp	ASSUMPTION; replaced in week 1
Future orders per retained customer (6 months)	3	5	8	ASSUMPTION
Ownly revenue per order (delivery fee)	₹30	₹30	₹30	SECONDARY
Contribution per order	₹5	₹10	₹15	UNKNOWN (delivery cost not public)

Calculation (per month)

Step	Formula	Conservative	Base	Upside
A. Severe failures	300,000 × f	2,250	4,500	9,000
B. GMV in failed orders	A × ₹300	₹6.75L	₹13.5L	₹27.0L
C. False deliveries prevented	A × 40% × OTP cut	270	900	2,520
D. Other failures neutralised	A × 60% × auto-refund effect	203	675	1,890
E. Failures prevented or neutralised	C + D	~473	1,575	4,410
F. Food value no longer lost to false delivery	C × ₹300	₹0.81L	₹2.70L	₹7.56L
G. Extra orders over 6 months (retention)	E × repeat gap × future orders	~142	1,575	~10,584
H. Extra GMV (6 months)	G × ₹300	₹0.43L	₹4.73L	₹31.75L
I. Extra Ownly revenue (6 months)	G × ₹30	₹4.3K	₹47K	₹3.18L
J. Extra contribution (6 months)	G × contribution	₹0.7K	₹15.8K	₹1.59L

Checks: C (base) = 4,500 × 0.4 × 0.5 = 900; D = 4,500 × 0.6 × 0.25 = 675; G = 1,575 × 0.20 × 5 = 1,575; conservative G = 473 × 0.10 × 3 ≈ 142; upside G = 4,410 × 0.30 × 8 = 10,584. ₹1L = ₹100,000. G, H, I and J accrue from one month's cohort of prevented failures, over the following six months. Rounding is to the displayed precision.

What the model says, honestly

- **Direct contribution is small.** Even in the upside case, extra contribution from retention is ~₹1.6L per monthly cohort per 10k orders/day. On direct financial return alone, the MVP would be hard to justify ESTIMATE.
- **Who bears F depends on today's refund policy.** If the customer currently absorbs a false delivery (no refund), F is a customer loss that becomes churn, not an Ownly saving. If Ownly refunds it, F is a direct saving. The split is UNKNOWN.
- **The real return is unmodelled:** store conversion (detailed 1★ reviews are what prospective users read), Rapido brand protection, and restaurant volume (GMV retained flows to restaurants that are deciding whether to keep the Ownly tablet on).
- **The two numbers that decide everything are f and the repeat gap.** Below f = 0.5% with a repeat gap under 5 pp, this is a CX fix, not a strategy.

> **PM DECISION · Size before you build**
> The opportunity cannot be reliably quantified from public data. The first deliverable is therefore two queries on existing order logs (failure rate and repeat gap), not a feature. That costs about two analyst-weeks and decides whether the rest of this plan proceeds.

13 Business → product outcome tree

The tree shows how fixing the root cause would connect to business results, and which link each part of the plan tests.

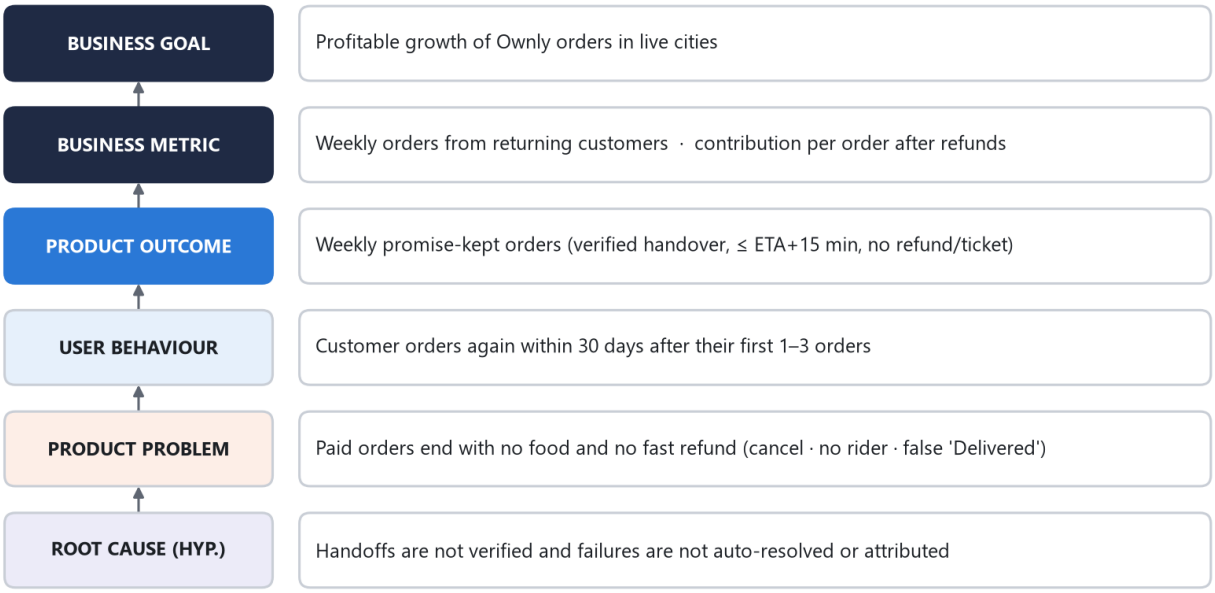


Exhibit 12. Outcome tree. The experiment tests the bottom three links; the sizing gate tests the link between the user behaviour and the business metric.

Link in the tree	What could break it	How we test it
Root cause → product problem	Most “not received” claims are abuse or confusion, not false deliveries	OTP pilot: do claims fall when handover is verified?
Product problem → behaviour	Customers whose orders fail still come back (price beats pain)	Sizing gate: repeat gap, failed vs clean
Behaviour → product outcome	Promise-kept orders rise but refunds or captain friction offset it	Guardrails: refund cost, captain acceptance
Product outcome → business metric	Contribution per order falls after refunds	System rule: stop if contribution after refunds drops > ₹3/order

14 Competitive analysis

This is not a general comparison. It looks only at the parts of the competitive picture that bear on the paid-no-food problem. Claims about competitors' internal workflows were **not** observed in this research and are marked REQUIRES VALIDATION; a structured walkthrough is proposed in Appendix F.

Dimension	Ownly	Swiggy / Zomato	Source · confidence
Customer price	Restaurant menu price, no platform fee, ~₹30 delivery fee	Platform fee ~₹15–18; menu mark-ups reported	SECONDARY press; reviews confirm Ownly is seen as cheaper
Restaurant economics	Zero commission (reported)	15–30% commission	SECONDARY
Delivery fleet	Ownly Captain app on Rapido's network; food can be a side gig between rides	Dedicated fleets with bag and handling standards	Ownly: reviews OBSERVATION; competitors: REQUIRES VALIDATION
Trust & recovery	Bot-first, email redirects, refunds stuck at "initiated"	Customers say incumbents support better (*"not supporting like Zomato and Swiggy"*)	Customer and partner reviews OBSERVATION; incumbent flows REQUIRES VALIDATION
Communication	Fast templated store replies (median ~14 min) that point to email	Not measured	Review metadata ACTUAL
Restaurant tooling	Menu and price edits need Ownly's team; alerts unreliable on tablets	Self-serve menus and analytics (general knowledge)	Partner reviews OBSERVATION
Discovery	1.1K est. organic visits/month; found as "rapido food delivery"	67.9M (Zomato) / 15.3M (Swiggy)	Semrush ACTUAL (modelled)
Distribution	Rapido super-app cross-sell	Own apps with large installed bases	SECONDARY

What competitors do better

- Resolve problems and support restaurants. Both customers and partners say so in their own words OBSERVATION.
- Run delivery with dedicated fleets and handling standards REQUIRES VALIDATION.
- Brand trust and search presence: ~1,000× Ownly's modelled organic traffic ACTUAL.

What competitors do worse

- Price. Customers explicitly contrast Ownly's pricing with incumbents' fees OBSERVATION.
- Restaurant take rate. Commission is their profit pool, so matching Ownly's zero-commission offer would hurt them INFERENCE.

Opportunity and realistic differentiation

Ownly cannot out-spend incumbents on brand, and does not need to beat them on speed. Two things it can realistically own:

1. **Fair price** (already working), which incumbents cannot easily copy without cutting into their profit.
2. **A verifiable handover** built on a habit Rapido's users and captains may already have. Rapido rides start with a code the rider shares REQUIRES VALIDATION: widely understood, but not verified in this research. If true, a delivery code is a native behaviour on Ownly's network rather than new friction.

> KEY INSIGHT · **Positioning**
>
> "The fair-price food app you can trust with your money." Fair price is working. Trust is the missing half, and it is cheaper for Ownly to build trust than for incumbents to copy fair pricing INFERENCE.

15 Solution space

Ten candidate interventions were generated against the validated problem and its suspected causes, before any was chosen. They include options that address symptoms, options that address causes, and options that look attractive but fail on second-order effects.

Value

Solution	Problem addressed	User value	Business value	Expected impact
S1 Delivery code (OTP) at handover	False “Delivered”	Certainty that nobody can claim food was delivered	Fewer false-delivery losses; makes “not received” claims verifiable	High on false deliveries
S2 Automatic refund rules	Money stuck after a failure	Money back in minutes, no ticket	Less support load; less fraud language	High on trust; low reach
S3 Restaurant missed-alert fallback (persistent alarm + automated call at 60 s)	Cancelled after payment	Fewer cancellations	Saves GMV; fewer partners blamed	Medium
S4 Captain-assignment timeout + honest ETA	No captain for 40–75 min	Option to cancel with a refund instead of waiting blind	Caps the worst waits	Medium
S5 Live human for active orders	Bot loop	Someone who can decide	Recovers some failures	Medium; cost grows with orders
S6 Prep-aware dispatch	Captain too early or idle	Better ETA accuracy	Less captain idle time	Medium; needs prep data
S7 Captain onboarding status + zone capacity gate	Captain applicants waiting a median 30 days	Applicants know where they stand	Healthier captain supply; fewer 1★ on captain app	Medium (supply side)
S8 Quick wins: reply copy with outcome, optional rating prompt, offer bug fix	Metric distortion; email redirects	Less irritation	Honest ratings	Low but cheap
S9 Cash on delivery	Payment anxiety	Pay later	Some conversion	Negative: more cancellations, captain cash risk
S10 SEO / discounts	Discovery	—	More traffic into failures	Low now

Feasibility and risk

Solution	Confidence	Eng.	Ops.	Dependencies	Risks / unintended consequences
S1 OTP	MEDIUM-HIGH	Medium	High	Captain app release; customer app	Captains decline food orders; codes asked for early over the phone
S2 Auto-refund	MEDIUM	Medium	Low	Payments, wallet, policy and legal	Abuse; refund cost rises before prevention takes effect
S3 Alert fallback	MEDIUM	Low	Medium	Telephony vendor	Partner annoyance; cost per call
S4 Assign timeout	MEDIUM	Low	Low	Dispatch events	More cancellations if the threshold is too tight
S5 Live human	MEDIUM	Medium	High	CX headcount	Cost scales with orders; treats symptom
S6 Prep-aware dispatch	LOW-MEDIUM	High	Medium	Prep-time data from partners	ETA regressions while it learns
S7 Onboarding status	MEDIUM	Medium	Medium	KYC vendors; supply planning	Exposes backlog publicly
S8 Quick wins	HIGH	Low	Low	None	Visible rating may dip (more honest)
S9 COD	LOW	Medium	High	Captain cash handling	Captains already ask for extra cash; more cancellations
S10 SEO / discounts	LOW	Medium	Low	Marketing budget	Accelerates exposure to failures

16 Solution prioritization

RICE was used as a first pass. Reach is an index (100 = all orders in a quarter); Impact 0.25–3; Confidence 0–1; Effort in person-months. All inputs are ESTIMATEs and are relative, not precise.

Solution	Reach	Impact	Conf.	Effort	RICE	Strategic weight	Tier
S8 Quick wins	100	0.5	0.8	0.5	80.0	Metric hygiene	Do now (no test)
S1 Delivery OTP	100	2	0.7	3	46.7	High · root cause	**MVP**
S3 Alert fallback	30	2	0.5	1	30.0	High · cancellations	Parallel test
S6 Prep-aware dispatch	100	2	0.4	6	13.3	High, needs data	Later experiment
S4 Assign timeout	10	2	0.6	1	12.0	Medium	**MVP (rule R2)**
S2 Auto-refund	10	3	0.7	2	10.5	Very high · trust	**MVP**
S5 Live human	15	2	0.5	3	5.0	Medium; costly	After MVP readout
S9 COD	20	1	0.3	2	3.0	Negative	Do not build
S7 Onboarding status	20	1	0.4	3	2.7	Supply health	Separate track
S10 SEO / discounts	5	0.5	0.3	3	0.25	Low now	Do not build

RICE = Reach × Impact × Confidence ÷ Effort. Example: S1 = 100 × 2 × 0.7 ÷ 3 = 46.7.

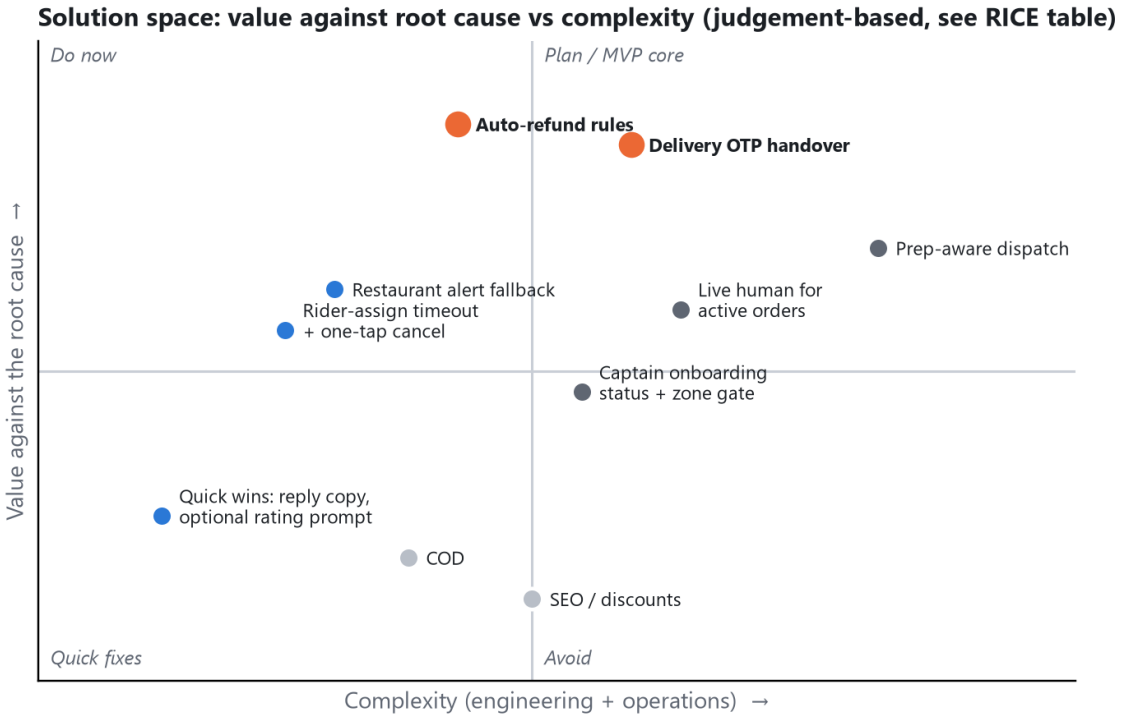


Exhibit 13. Value against the root cause vs complexity. Positions are judgement, consistent with the RICE inputs.

Why the framework does not make the decision

- **RICE ranks cheap cosmetic fixes first** (S8 = 80) because they reach everyone. They should ship, but they do not touch the root cause.
- **RICE ranks auto-refund low** (10.5) because few orders fail. Yet it addresses the single moment that creates fraud language, and it is what makes a delivery code visible as protection to the customer.
- **S1 and S2 are a system, not two features.** The code makes generous auto-refunds affordable, because "not received" claims become checkable. The auto-refund makes the code meaningful to the customer. Shipping one without the other weakens both.

- **S3 tests a different cause** (restaurant alerts) and needs no captain-app change, so it runs as a separate, cheaper experiment rather than inflating the MVP.

> **PM DECISION · Selected: S1 + S2 (incl. S4's timeout as rule R2) as the MVP**
>
> It is the smallest intervention that tests the core hypothesis: verifying the handoff and resolving failures automatically reduces paid-no-food losses and protects repeat ordering. It uses a habit Rapido's network may already have. **Deferred:** S5 (until residual ticket volume is known), S6 (needs prep data), S7 (separate supply track). **Rejected:** S9 COD (worsens the core failure), S10 SEO/discounts (pours users into a leak).

17 MVP definition

Ownly Promise v1	1 city	≥ 4 weeks
Verified handover + automatic refunds	Zone-randomised pilot	Minimum read time

Element	Definition
MVP objective	Test whether verifying the handover and auto-resolving eligible failures reduces “not received” claims and narrows the repeat gap for customers who hit a failure
Target user	Prepaid customers in pilot zones; primary focus on customers in their first 1–3 orders
Core problem	Paid, no food, and nobody fixes it
Value proposition	“Your food, or your money back in minutes.” (Shown only in treatment zones; not marketed until proven)
In scope	4-digit delivery code; captain fallback (photo + GPS ≤ 75 m) when the customer is unreachable; refund rules R1–R3; cause code and owner on every failure; refund receipt screen
Out of scope	Live-agent support redesign; chatbot rebuild; COD; prep-aware dispatch; captain bags and uniforms; merchant self-serve tools; loyalty; new cities; restaurant alert fallback (separate test)

User flow

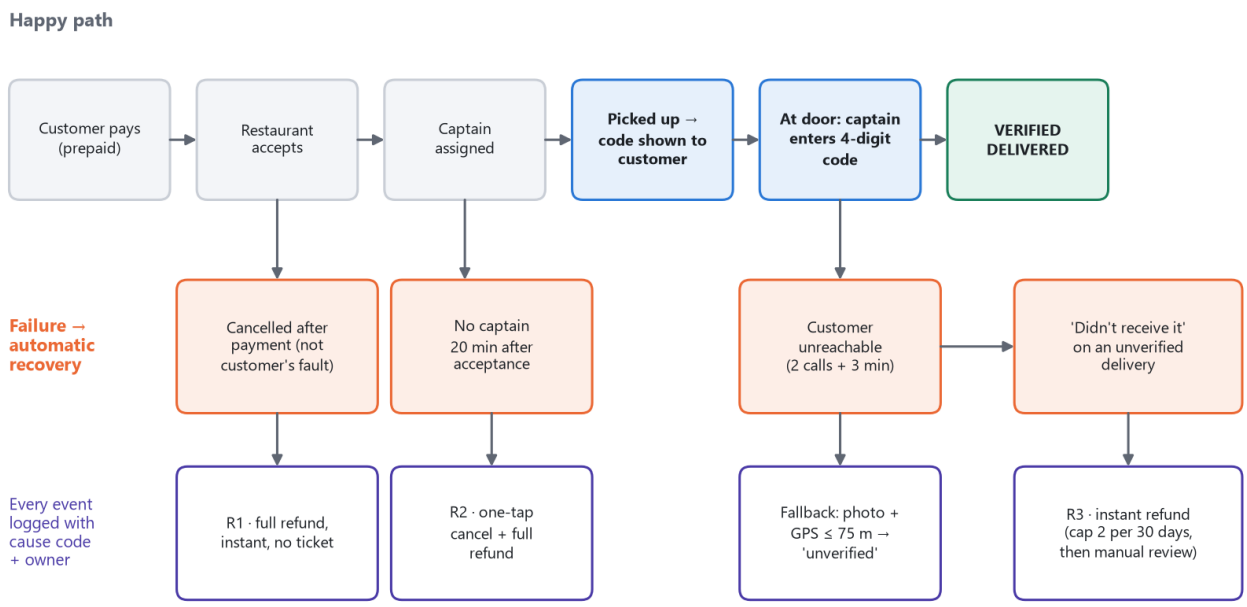


Exhibit 14. MVP flow: happy path, failure states and automatic recovery.

Key functionality

Rule / feature	Trigger	Action
Delivery code	Order picked up	Show 4-digit code on the tracking screen and in a push notification. Captain cannot mark delivered without it
Fallback	Customer unreachable: 2 calls + 3 min at the drop point	Photo + GPS within 75 m → status “Delivered – unverified”
R1	Cancelled after payment for any non-customer reason	Full refund immediately, no ticket

Rule / feature	Trigger	Action
R2	No captain 20 min after the restaurant accepts	Offer one-tap cancel with full refund, or keep waiting with an updated ETA
R3	Customer reports “not received” on an unverified delivery	Instant refund; capped at 2 per customer per 30 days, then manual review
Ledger	Any cancellation, refund or unverified delivery	Record cause code (restaurant / captain / platform / customer), owner and timestamps

User stories and acceptance criteria

User story	Acceptance criteria (Given / When / Then)
As a customer, when my food arrives I share a code, so nobody can claim it was delivered when it wasn't	Given an order is picked up, when I open tracking, then I see a 4-digit code and the copy “Share only when food is in your hands”. When the captain enters the correct code, the status becomes “Delivered – verified” within 2 s
As a customer whose order was cancelled after I paid, I see my money back without asking	Given a non-customer cancellation, when it is recorded, then a refund is issued within 60 s and the receipt shows amount, destination, time and reason
As a customer with no captain after 20 min, I choose whether to wait or cancel	Given acceptance + 20 min with no captain, then I get a push and an in-app card with “Cancel · refund ₹X” and “Keep waiting (new ETA)”
As a captain, I can finish a delivery when the customer doesn't answer	Given 2 unanswered calls and 3 min at the drop point, when I take a photo within 75 m of the pin, then the order closes as “Delivered – unverified”
As an ops analyst, I can see every failure with its cause	Every cancellation, refund and unverified delivery has a cause code and owner; a daily export reconciles to order counts within 0.5%

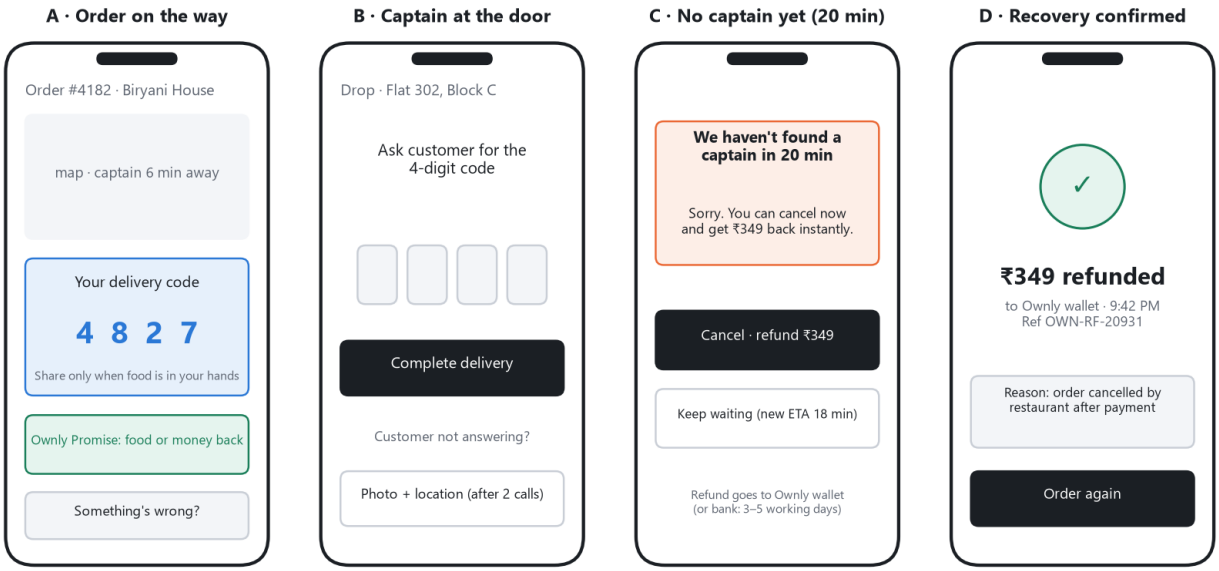
Edge cases

- Order placed for someone else → share the code to the recipient's number via SMS or WhatsApp.
- Gated community or guard handover → the guard reads out the code; otherwise fallback.
- Customer's phone is dead → fallback (photo + GPS).
- Captain reassigned mid-trip → the code stays valid; the event is logged.
- Captain asks for the code before arriving (“I'm downstairs”) → GPS distance is checked when the code is entered; copy warns the customer.
- Partial or wrong items → out of scope; existing flow.
- Repeat “not received” claimants → R3 cap; device and account linking.

Dependencies and risks

- **Captain app release** (critical path). Whether Ownly controls the Ownly Captain app end-to-end, or needs Rapido's captain app, is REQUIRES VALIDATION.
- **Payments** for instant wallet credit; **Legal** for refund terms; **CX** for the policy change.
- **Top risk:** captains quietly decline food orders because the code removes shortcuts. This would turn false deliveries into no-captain failures. It is treated as a hard guardrail (Section 20).

18 Product experience & wireframes



Low-fidelity wireframes for discussion. Amounts, order IDs and times are illustrative placeholders, not real data.

Exhibit 15. Key screens: entry point (A), core interaction (B), failure state (C), recovery/success state (D).

Screen	Purpose	UX decision and reasoning
A · Order on the way (entry point)	Introduce the code at the moment it matters	Code shown large on the tracking screen, **not only in SMS**, so it is available offline and visible to the person waiting. Copy explains *why* before asking: "Share only when food is in your hands". That turns the code into protection rather than friction.
B · Captain at the door (core interaction)	Make verification as quick as starting a ride	Mirrors a familiar 4-box code entry pattern (assumed similar to ride start; REQUIRES VALIDATION). The fallback is visible but secondary, unlocked only after 2 calls, so it is not the default path.
C · No captain yet (failure state)	Give control back at the worst moment	Shown proactively at 20 min, before the customer has to search for help. Two clear choices; the refund amount is stated, not "initiated". A new ETA is shown if they choose to wait.
D · Recovery confirmed (success / recovery state)	Replace uncertainty with a fact	A refund is shown as a completed fact with amount, destination, time, reference and reason, which directly answers the "refund stuck in initiated" complaint. Ends with "Order again" to support the repeat behaviour.

KEY INSIGHT · Design principle

Every screen answers one of three questions the reviews show customers cannot answer today: *Is my food really with me? Where is my money? What happens next?* Visual design is deliberately plain; the work is in the states and the copy.

Languages: English, Hindi, Kannada (pilot), with Telugu for Hyderabad. Accessibility: code digits ≥ 24 pt; not conveyed by colour alone.

19 PRD — Ownly Promise v1

Section	Content
1. Problem	Prepaid orders sometimes end with no food (cancelled after payment, no captain, false “Delivered”), and refunds require chasing. These customers rate ~1★ and use fraud language. Frequency is unknown (sizing gate first).
2. Objective	In pilot zones: reduce “not received” claims per 1,000 delivered orders by ≥40%; refund 90% of eligible failures within 5 minutes; keep the 30-day repeat rate of failed-order customers within 5 pp of clean-order customers.
3. Target users	Customers (primary); captains (must adopt the code); ops/CX (consume the ledger).

4. User stories

See Section 17 (five stories with acceptance criteria).

5. Functional requirements

ID	Requirement	Priority
FR-1	Generate a 4-digit code per order at pickup; show it on tracking and in a push notification	P0
FR-2	Captain app blocks “Delivered” until the code is entered; 3 wrong attempts lock the order and alert ops	P0
FR-3	Fallback: after 2 calls + 3 min at the drop point, photo + GPS ≤ 75 m → “Delivered – unverified”	P0
FR-4	R1: non-customer cancellation after payment → full refund, no ticket	P0
FR-5	R2: no captain 20 min after acceptance → push + in-app card offering one-tap cancel and refund	P0
FR-6	R3: “not received” on an unverified delivery → instant refund; cap 2 per customer per 30 days, then manual review	P0
FR-7	Refund defaults to Ownly wallet (instant); source refund on request with the bank's expected time shown	P0
FR-8	Cause code + owner + timestamps on every cancellation, refund and unverified delivery; daily export	P0
FR-9	Refund receipt screen (amount, destination, time, reference, reason)	P0
FR-10	“Something's wrong?” entry on active and delivered orders that runs R1–R3 before routing to a person	P1
FR-11	Treatment-only “Ownly Promise” badge at checkout and on tracking	P1

6. Non-functional requirements

- Code validation ≤ 2 s at p95 on 3G; works with intermittent connectivity (queued validation with a signed token).
- Refund initiation ≤ 60 s at p95 after a rule fires; idempotent (no double refunds).
- Ledger completeness ≥ 99.5% of failure events; immutable audit log; dual approval for manual refunds above ₹500.
- Codes are never written to SMS logs in plain text beyond the delivery window. Photos retained 30 days.
- Localised copy (EN/HI/KN); accessible contrast and sizes.

7. Acceptance criteria

Per user story (Section 17), plus: in internal testing, 100% of orders have a code at pickup; 0 orders reach “Delivered – verified” without a matching code; refund latency meets the NFR on 200 staged failures.

8. Edge cases

See Section 17: recipient differs from orderer, gated communities, dead phone, reassignment, early code requests, partial orders, repeat claimants.

9. Dependencies

Captain app release cycle (critical path); payments/wallet; telephony (for the parallel alert test); CX policy; legal review of refund terms; data engineering for the ledger.

10. Risks

Captain adoption and acceptance; refund abuse; refund cost rising before prevention takes effect; release delays. Details and mitigations in Section 22.

11. Analytics requirements

Events and properties in Section 21. All events carry order_id, zone_id, experiment_arm, city and timestamp.

12. Rollout plan

Internal → small cohort → zone-randomised experiment → expansion → full rollout, gated at each step (Section 23).

20 Experiment design

PM DECISION · Gate A comes first: E0 sizing (weeks 1–2, not an A/B test)

Pull (1) the severe-failure rate per 1,000 orders by type and (2) the 30-day repeat rate of customers whose first order failed vs those whose first order was clean.

Proceed at full scope if severe failures are ≥ 1% of orders **or** the repeat gap is ≥ 10 pp. **Stop and pivot** to the retention/selection hypothesis (H9) if failures are < 0.5% **and** the gap is < 5 pp. In between: ship R1 + ledger + quick wins only, and re-read in 30 days.

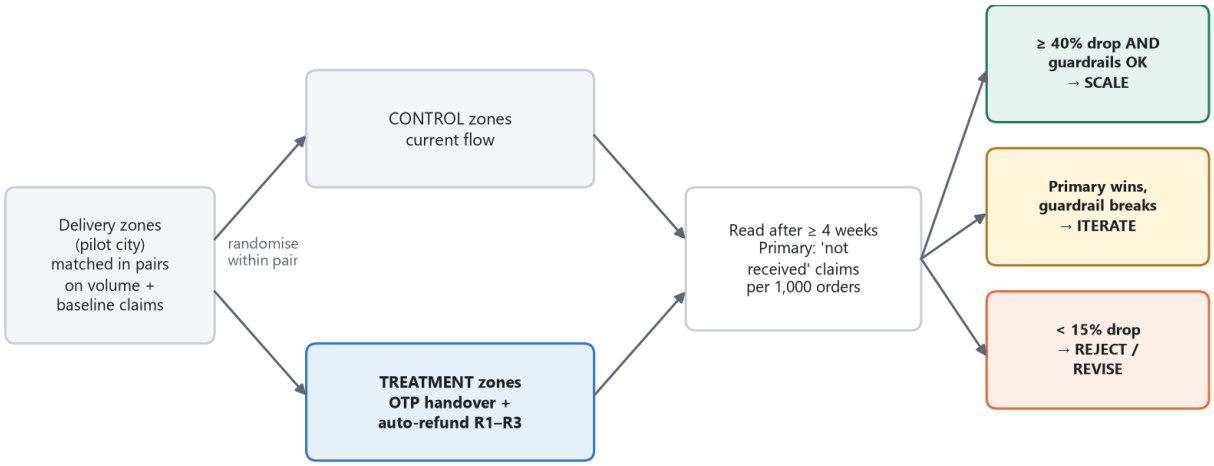


Exhibit 16. E1 design: zone-randomised pilot with pre-agreed decision rules.

Field	E1 · Ownly Promise v1
Hypothesis	If the handover requires the customer's code and eligible failures auto-refund, then “not received” claims per 1,000 delivered orders fall by ≥ 40%, and failed-order customers' 30-day repeat moves closer to clean-order customers', because false deliveries become impossible to hide and money-loss moments resolve in minutes
Type	Cluster-randomised controlled trial by delivery zone (not by user: captains and restaurants are shared within a zone, so user-level randomisation would leak)
Control	Current flow in matched zones
Treatment	Code at handover + R1–R3 + receipt screen
Audience	All prepaid orders in ≥ 6 matched zone pairs in the pilot city (Bengaluru, subject to E0 volume)
Sample	Illustrative: baseline claim rate 1.5%, target 50% relative cut (to 0.75%), α = 0.05, power 0.8 → n ≈ 7.84 × [0.015×0.985 + 0.0075×0.9925] ÷ 0.0075^2 ≈ **3,100 orders per arm** before clustering. With a design effect of 2–3 → **~6,200–9,300 per arm**. Recompute with the real baseline from E0
Duration	≥ 4 weeks (two full weekly cycles plus captain adaptation); repeat-rate read at day 30–45 after the pilot start
Primary metric	“Not received” claims per 1,000 delivered orders
Secondary	Delivered-without-GPS-match %; refund latency; refund cost per 1,000 orders; 30-day repeat of failed-order customers; fraud/scam language per 1,000 orders in post-order feedback
Guardrails	Captain food-order acceptance (≤ 5% relative drop); median handover time added (≤ 45 s); fallback usage (≤ 15% of deliveries); refund abuse rate (≤ 2% of auto-refunds); delivery time; contribution per order after refunds (≤ ₹3 drop)
Success threshold	≥ 40% relative drop in primary metric with all guardrails intact
Failure threshold	< 15% drop, or captain acceptance down > 5%
Confounders	Rain or festivals; city-wide promos; Rapido incentive changes; novelty effects. Mitigated by matched pairs and difference-in-differences against the pre-period

Decision rules

If…	Then…
Primary improves ≥ 40% AND guardrails healthy	**Scale**: city-wide in two waves, then the next city
Primary improves but captain acceptance drops > 5%	**Iterate**: test a targeted compliance bonus in low-supply hours only; simplify the fallback; re-run 2 weeks
Primary improves but refund abuse > 2%	**Iterate**: tighten R3 eligibility (device linking, lower cap)
Primary improves 15–40%	**Iterate**: inspect fallback share and early-code requests; extend 2 weeks
Primary < 15% improvement	**Reject / revise**: most claims were not false deliveries; redirect to cancellations (E2) or H9

E2 · Restaurant alert fallback (parallel, independent)

Randomised by restaurant. Treatment: persistent alarm + automated call if an order is unacknowledged at 60 s, and an ops call at 3 min; never auto-cancel silently. **Primary:** restaurant-caused cancellations after payment. **Guardrail:** partner opt-outs and complaints. Two weeks. It needs no captain-app change, so it can start during the E1 build.

21 Metrics & instrumentation

Layer	Metric	Definition
North Star	**Weekly promise-kept orders**	Orders delivered with a verified handover, ≤ ETA + 15 min, and no refund or ticket. Resistant to gaming: discounts can buy orders and prompts can buy stars, but neither creates kept promises
Primary KPI (pilot)	"Not received" claims per 1,000 delivered orders	Tickets or R3 triggers tagged not-received ÷ delivered orders × 1,000
Retention KPI	30-day repeat rate, failed vs clean first order	Share of customers placing another order within 30 days, split by whether their first order failed
Input	Code match rate; fallback share; time to captain assignment; post-payment cancellation rate by cause; refund latency p50/p95; share auto-resolved	Leading indicators the team can move directly
Output	Promise-kept orders; repeat rate; contribution per order after refunds; fraud/scam mentions per 1,000 orders	Business-facing results
Guardrail	Captain food-order acceptance; handover time added; refund cost per 1,000 orders; refund abuse rate; delivery time; partner opt-outs	Must not deteriorate beyond thresholds in Section 20

Analytics events (only those the MVP needs)

Event	Trigger	Key properties	Metric supported
order_paid	Payment success	order_id, amount, customer_order_index, zone_id, arm	Denominators; first-order segment
order_accepted	Restaurant accepts	restaurant_id, accept_latency_s	Time to accept
captain_assigned	Captain matched	captain_id, assign_latency_s	Time to assign; R2 trigger
assign_timeout_shown	20 min without a captain	minutes_waited, choice (cancel/wait)	R2 uptake
picked_up	Captain collects food	captain_id, wait_at_restaurant_s	Idle time
delivery_code_shown	Code displayed to customer	surface (tracking/push)	Code exposure
delivery_code_entered	Captain submits code	success, attempt_no, gps_distance_m	Code match rate; early-entry checks
fallback_delivery	Photo + GPS fallback used	photo_id, gps_distance_m, calls_made	Fallback share
order_delivered	Order closed as delivered	verified (bool), eta_delta_min	North Star; ETA accuracy
order_cancelled	Any cancellation	cause_code, actor, post_payment (bool)	Cancellation rate by cause
issue_reported	Customer uses "Something's wrong?"	issue_type, order_state	Primary KPI
refund_rule_fired	R1/R2/R3 eligibility met	rule_id, amount	Auto-resolution share
refund_issued	Refund executed	amount, destination, latency_s, rule_id or manual	Refund latency; cost
repeat_order	Customer's next order_paid	days_since_prev, prev_order_failed (bool)	Retention KPI

22 Risks & second-order effects

Type	Risk	Prob.	Impact	Mitigation
Marketplace	Captains decline food orders because the code removes the option to abandon; false deliveries turn into no-captain failures	Medium	High	Hard guardrail; code flow mirrors ride start; fast fallback; targeted bonus only in low-supply hours if needed
Abuse / fraud	Customers claim “not received” on unverified deliveries; captains ask for the code early by phone	Medium	Medium	R3 cap + device linking; GPS distance check on code entry; customer copy warning
Financial	Refund cost rises before prevention works	Medium	Medium	Budget envelope agreed up front; ledger attributes cost by cause; chargebacks to the party at fault from week 6
Operational	Ops desk overloaded by escalations and lockouts in week 1	Medium	Low	Staff peak hours; auto-unlock after review
Technical	Captain app release delays; poor connectivity at doors	Medium	High	Ship R1, ledger and E2 first (no captain-app change); offline code validation token
User	Extra step annoys customers; recipients without the app	Low	Medium	Code in push + SMS to recipient; measure handover time
Adoption	Captains skip to fallback by default	Medium	Medium	Fallback unlocks only after 2 calls + 3 min; fallback share is a guardrail
Business / brand	Promising “food or money back” before it works backfires	Low (if gated)	High	No marketing of the Promise until Gate B; badge only in treatment zones
Partner	Restaurants resent automated calls or chargebacks	Medium	Medium	Share data before chargebacks; dispute flow; chargebacks only after week 6
Cannibalisation	None on GMV; may shift CX headcount plans	Low	Low	—

Second-order effects

- **Positive loop (expected):** verified handovers → fewer paid-no-food orders → fewer fraud reviews → more repeat orders → more orders per restaurant → partners keep the tablet on and accept faster → fewer cancellations HYPOTHESIS.
- **The loop may currently run in reverse:** partners report “one order a day”, may deprioritise the tablet, miss alerts, and cause cancellations that suppress demand INFERENCE.
- **Liability attribution changes behaviour on all sides.** Once failures are charged to the party at fault, marginal partners may leave. That improves quality but reduces listed selection.
- **Inversion: how to guarantee failure.** Market the Promise before the code works; pay captains per code so costs explode; set loose refund rules that spread on WhatsApp; ignore restaurant-side cancellations so the refund bill grows. The plan is built to avoid each.

23 Launch & rollout plan

Phase	Objective	User group	Metrics	Exit criteria	Decision
0 · Sizing (wk 1–2)	Size the problem	Historical orders	Failure rate; repeat gap	Gate A thresholds (Section 20)	Go / reduced scope / pivot
1 · Internal validation (wk 3–4)	Prove the flow works	Employees + 20 trained captains	Code success rate; refund latency on staged failures	0 unverified deliveries without code; NFRs met	Proceed to cohort
2 · Small cohort (wk 5)	Catch real-world issues	1 zone pair	Fallback share; handover time; lockouts	Fallback < 25%; no P0 bugs	Fix, then randomise
3 · Experiment (wk 6–9)	Measure causal impact	≥ 6 matched zone pairs	Primary, secondary, guardrails	Pre-registered thresholds	Scale / iterate / reject (Gate B)
4 · Expansion (wk 10–13)	Confirm at scale	50% of city	Same, plus contribution per order	Effects hold within 20% of pilot	City-wide
5 · Full rollout (Q2+)	Make it the default	Full city → next city	North Star; repeat rate	Guardrails stable 4 weeks	Market the Promise; chargebacks live

24 Expected impact & decision framework

IMPORTANT ASSUMPTION · Nothing here has happened

Ownly Promise v1 has not been built or tested. Every result in this section is EXPECTED, conditional on the sizing gate.

Question	Answer
Expected impact	EXPECTED 40–60% fewer “not received” claims in treatment zones; 90% of eligible failures refunded within 5 min; a narrower repeat gap for failed-order customers. Direct financial value per 10k orders/day ranges from ~₹0.8L to ₹7.6L/month in avoided food loss (Section 12), and depends entirely on the real failure rate
What we expect to learn	The true severe-failure rate and its causes; whether false deliveries are captain shortcuts or customer abuse; whether faster recovery changes repeat behaviour; whether captains accept the extra step
What would prove the hypothesis	Claims fall ≥ 40% with guardrails intact, **and** the repeat gap narrows for treated failed-order customers
What would disprove it	Claims barely move (most claims were not false deliveries); or claims fall but repeat behaviour does not change (failures were not the retention brake); or E0 shows failures < 0.5% with no repeat gap
If successful	Scale city-wide, then Hyderabad; switch on liability chargebacks; start the live-human test for residual issues; then make the Promise a public brand claim
If unsuccessful	Keep the ledger and alert fallback (useful regardless); stop OTP work; redirect the squad to H9 (retention/selection) or to cancellation causes if E2 shows a stronger effect

25 Future iterations

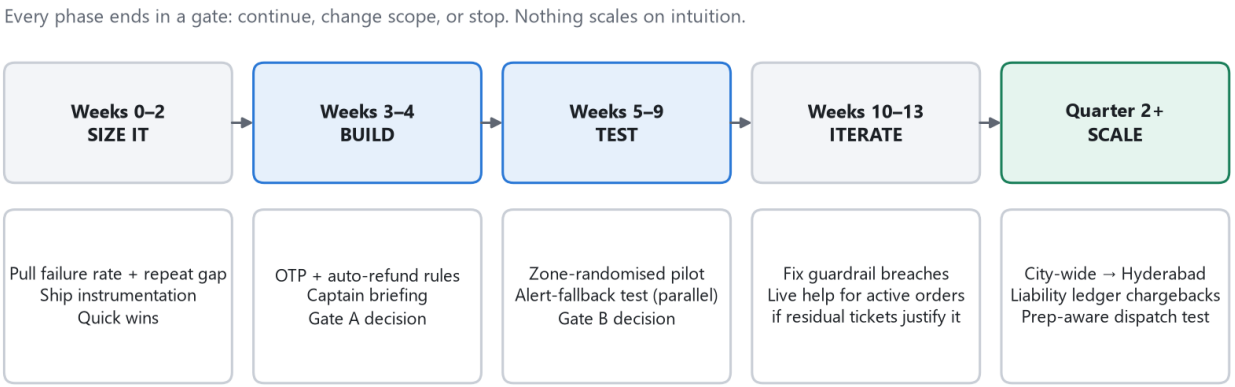


Exhibit 17. Roadmap. Each step depends on what the previous gate learns.

Next step	Only if…	Why it follows
Live human for active orders	E1 shows meaningful residual tickets after auto-rules	Automation should absorb the predictable cases first
Liability ledger chargebacks	Cause codes are accurate (audit ≥ 95%)	Makes generous refunds affordable and aligns incentives
Prep-aware dispatch	Prep-time data is reliable and early dispatch shows up in idle-time data	Attacks no-captain and idle-captain failures at source
Captain onboarding status + zone capacity gate	Dispatch data confirms allocation, not headcount, is the gap	Stops collecting documents from applicants Ownly cannot activate
Partner self-serve menu and payouts	Partner retention becomes the binding constraint	Removes the manual queue on the restaurant side
Public “Promise kept” rate	Rate is consistently high for 8+ weeks	Turns reliability into a differentiator

Deliberately absent: AI chat support, recommendation engines, loyalty schemes. None addresses the validated problem.

26 Key learnings

Area	Learning
The user	Customers come for price and forgive slowness; they do not forgive losing money. The emotional break point is “I paid and nobody helped”, which they express as “fraud”.
The product	The weakest part of the product is not a screen; it is the absence of proof at handoffs and of automatic recovery. Every side, including captains and restaurants, is stuck in an invisible queue.
The business	Zero commission makes reliability existential: there is little margin to absorb failures. Direct ROI from fixing them is modest; the value is retention, trust and Rapido's brand.
Assumptions disproven	“Failed recovery costs a full star” (does not hold for severe failures); “94% of helpful votes sit on complaints” (true, but n = 32); “not enough captains” (contradicted by idle and paused-onboarding signals; reframed); “SEO is a growth lever” (distribution runs through Rapido).
What remains uncertain	Failure rate per order; repeat gap; causes of cancellations; who controls the captain app; competitor recovery flows; iOS and non-reviewer sentiment.
Next PM decision	Request two queries (severe-failure rate and repeat gap) and decide at Gate A on day 14. Build nothing irreversible before then.

PM DECISION · Final quality check

- **Is the problem proven?** Its existence and severity are, through multiple sources. Its size is not, which is why the plan starts with sizing.
- **Could something else explain the evidence?** Yes: abuse, H9 (weak retention regardless), restaurant-side causes. Each has a test.
- **Is the MVP small and testable?** Two mechanisms plus instrumentation, one city, pre-registered thresholds.
- **Any invented facts?** No internal metrics are claimed; every number is labelled; four earlier claims were corrected (Appendix C).

27 Appendix

A. Customer review theme table (n = 1,000; 493 negative)

Theme	Journey stage	All mentions	In 1–2★	% of 1–2★	Avg ★
Support unresponsive	Service recovery	247	241	48.9%	1.11
Rider / delivery partner	Last mile	131	115	23.3%	1.43
Refund problem	Service recovery	119	113	22.9%	1.15
Delivery delay / ETA	Last mile	133	107	21.7%	1.73
Not delivered / false “Delivered”	Last mile	107	104	21.1%	1.07
Fraud / scam (dictionary)	Brand outcome	90	88	17.8%	1.11
Fraud / scam (strict wording)	Brand outcome	76	74	15.0%	1.12
Order cancelled	Restaurant acceptance	61	61	12.4%	1.00
Price / no-fee value (+)	Value driver	130	42	8.5%	3.70
Food quality / hygiene	Order receipt	62	40	8.1%	2.40
Missing / wrong items	Order receipt	42	39	7.9%	1.24
Payment / COD	Order & pay	40	33	6.7%	1.62
Competitor mention	Brand outcome	57	31	6.3%	2.84
App bugs / forced rating	Order & pay	30	25	5.1%	1.63
First-order experience	Segment	22	16	3.2%	2.05
Offer / promo mismatch	Discover & offer	18	15	3.0%	1.56
Restaurant selection	Value driver	15	10	2.0%	2.40
Packaging / spillage	Order receipt	8	8	1.6%	1.00

Method: case-insensitive keyword matching (dictionary in the source workbook), multi-tag. Strict fraud wording = fraud|froud|scam|fake|cheat|chor|loot (excludes “trust”, “trap”, “steal”).

B. Captain and restaurant app summaries

Captain app (n = 376, avg 2.63★; 223 negative)	Neg. mentions	% of neg.	Avg ★
Onboarding / verification delay	131	58.7%	1.34
Brand trust / “don't download”	67	30.0%	1.06
Support unresponsive	43	19.3%	1.59
App stability / login bugs	34	15.2%	1.71
Incentive & payout trust	21	9.4%	2.70
Order flow / dispatch	13	5.8%	1.88
City not live	5	2.2%	2.90

Burst-day 5★: 59 of 144 5★ reviews; organic average excluding them 2.19★. Developer reply rate 0%. Applicants 150 (40%), active captains 50.

Restaurant app (n = 105, avg 4.09★; organic 3.59★)	Problem reviews	Theme detail
Onboarding & activation	8	Documents re-requested; onboarding > 1 month; GSTIN field issue

Restaurant app (n = 105, avg 4.09★; organic 3.59★)	Problem reviews	Theme detail
Order alerts & cancellations	4	Ring not firing; background app killed; auto-cancel
Store visibility & availability	4	Live but not visible; cannot go online
Support responsiveness	4	Follow-ups unanswered
Menu & catalogue control	3	Cannot edit items, prices or photos
Rider & dispatch / demand / payouts / other	10	Early riders; low orders; payout mismatch

C. Corrections to prior analysis

Prior claim	Re-check	Status
“Failed recovery costs a full star” (2.33★ vs 1.07★)	Severe failures are ~1★ with or without support complaints; the gap comes from milder failures and false positives	REJECTED
“94% of helpful votes sit on negative reviews”	Arithmetically true (30 of 32), but the base is 32 votes across 1,000 reviews	Not meaningful
Fraud theme = 88 negatives	Dictionary includes “trust”, “trap”, “steal”; strict wording gives 74	Revised
Delay theme = 107 negatives	Dictionary includes “on time”, “punctual”; 21 tagged reviews contain on-time phrases, 15 of them 4–5★	Inflated
Captains are Rapido-app captains (OTP depends on Rapido)	Data shows a separate Ownly Captain app	REQUIRES VALIDATION
Quote: “since order is delivered we cannot do anything”	Not found verbatim in the scraped reviews	Excluded

D. SEO detail (Semrush, India desktop, 5 Oct 2026)

- Traffic ~1.1K/month; 163 keywords; branded 155 vs non-branded 931; keywords appear only from ~Jun 2026.
- 63.5% of traffic goes to one restaurant brand page (ownly.food/app/brand/BR430482).
- Ranks #1 for “rapido food delivery bangalore” (320/mo) and #3 for “rapido food delivery” (720), “rapido food app” (480), and misspellings “only app” (720), “onely”.
- Organic competitors listed (mrchamparan.com, dojo.app, menew.in) are not real competitors, which confirms how thin the footprint is.
- The “Top Opportunities” panel (“food open near me”, 823K) uses the **US** database and does not apply to an India-only service.

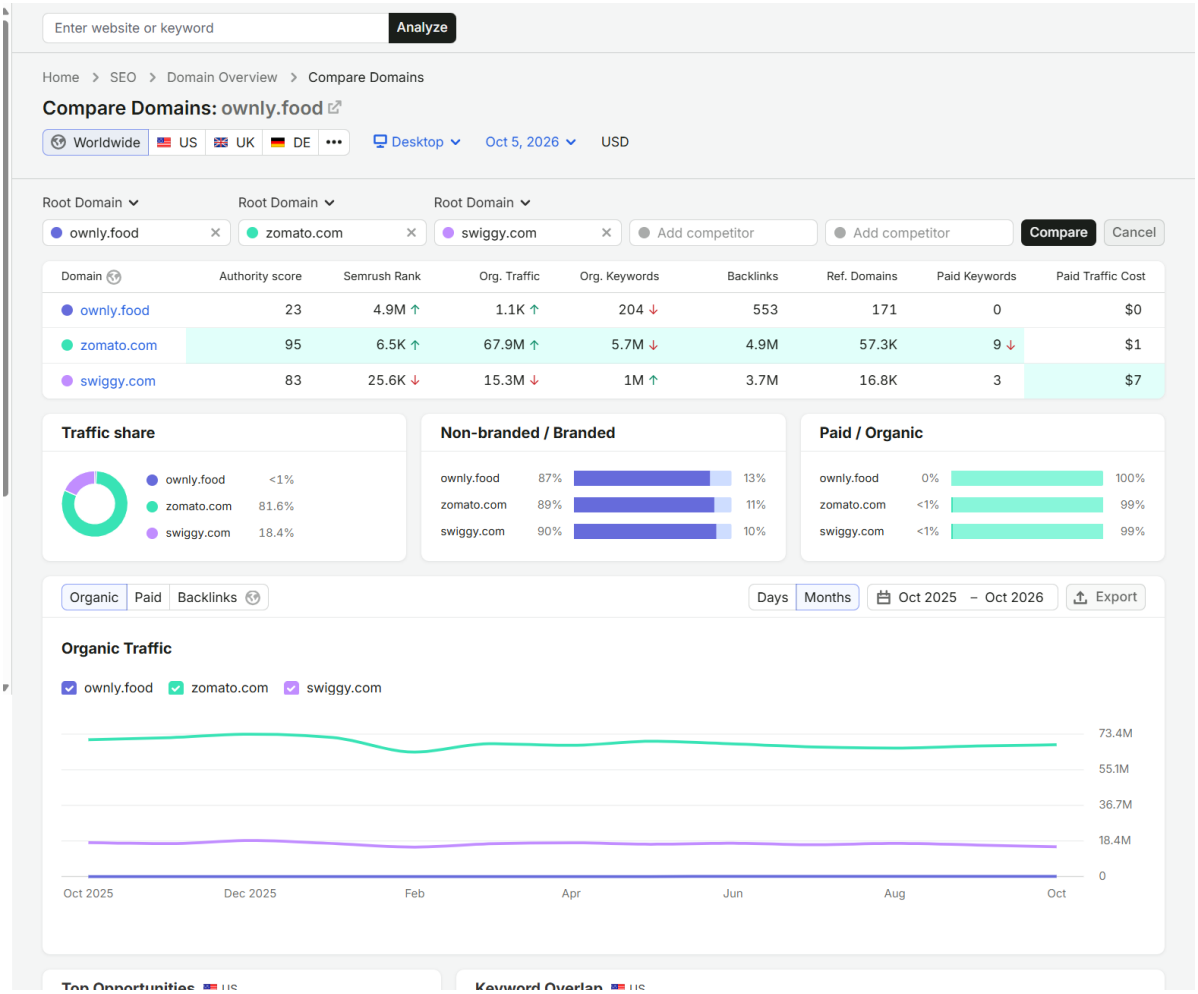


Exhibit A1. Semrush Compare Domains (source screenshot, cropped).

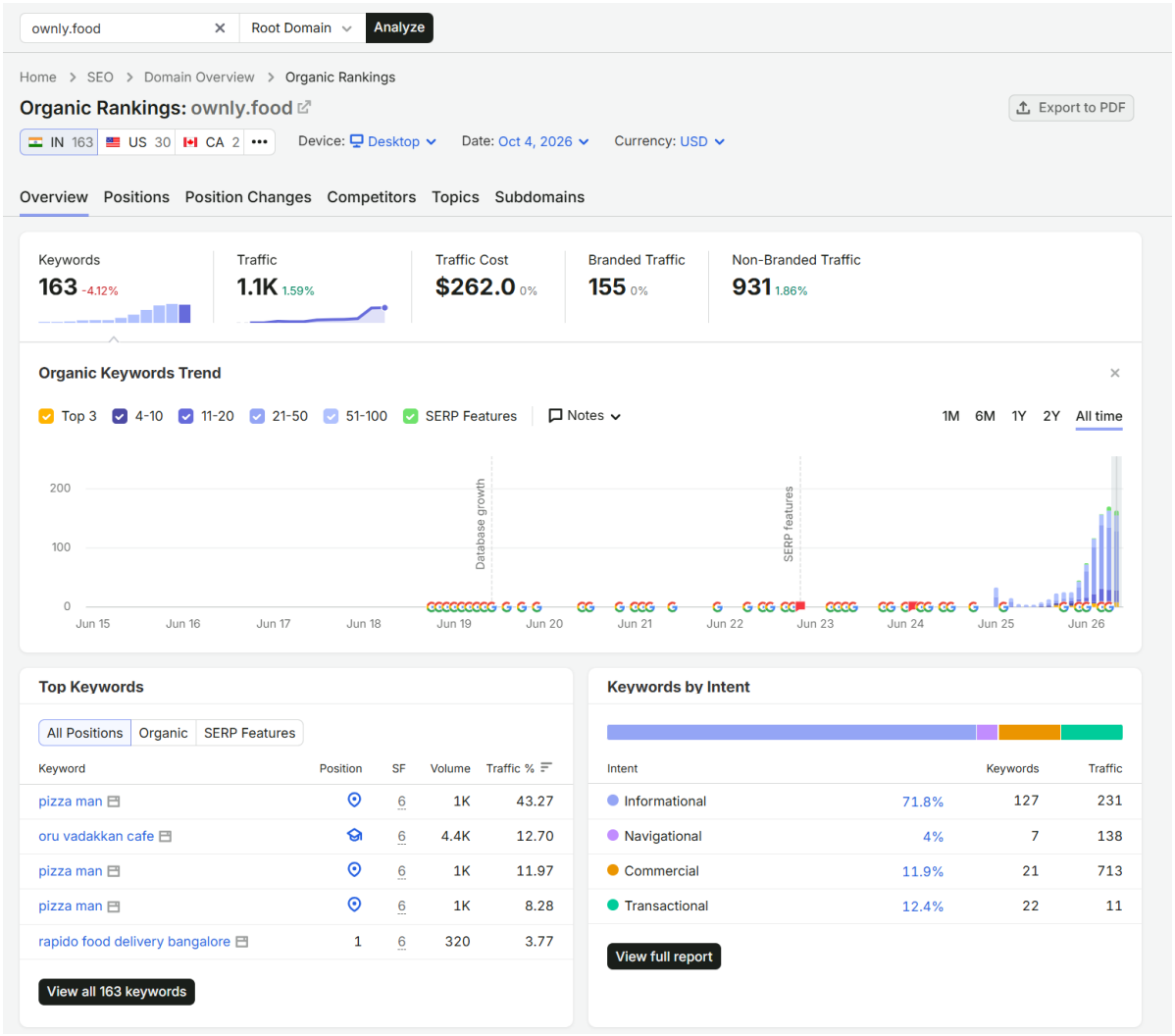


Exhibit A2. Semrush Organic Rankings overview (source screenshot, cropped).

E. Calculations and assumptions

Opportunity model: Section 12 (all formulas shown, with arithmetic checks). Sample size: n per arm = $(z_{1-\alpha/2} + z_{1-\beta})^2 \times [p_1(1-p_1) + p_2(1-p_2)] \div (p_1 - p_2)^2 = 7.84 \times 0.02222 \div 0.00005625 \approx 3{,}097$, rounded to 3,100; × design effect 2–3. RICE: Section 16.

#	Assumption	If wrong	How we check
A1	Severe failures ≥ ~1% of orders	It is a CX fix, not a strategy	E0 query
A2	Failed-order customers churn materially more	Reliability isn't the brake (H9)	E0 cohort query
A3	Ownly can change the captain app quickly	Fallback-only MVP; slower	Confirm app ownership
A4	Captains accept the code without large drop-off	No-captain failures rise	E1 guardrail
A5	Refund abuse stays manageable	Leakage	Caps; audit
A6	Fault can be attributed from event logs	Liability can't be shifted	Ledger audit
A7	AOV ~₹300; contribution ₹5–15/order	Sizing changes	Finance data

F. Research plan (not conducted; questions only)

No interviews or surveys were run for this study. The following is proposed to close the gaps:

Study	Sample	Key questions
Customer interviews	12 failed-order + 6 clean-order customers	What happened? How did you get your money back? Did you order again, and why or why not? Would a delivery code feel reassuring or annoying?
Captain interviews	15 captains, mixed tenure	When and why are food orders abandoned or marked early? How do busy-ping rules affect which orders you accept? Would you accept a code step like at ride start?
Restaurant interviews	12 high-cancellation restaurants	Why are alerts missed (device, background, staffing)? When do captains arrive relative to food readiness?
Usability test	8 sessions	Can a customer get from “my order is wrong” to resolution in under 60 s today? With the prototype?
Competitor walkthrough	5 orders each on Swiggy and Zomato	How do their handover verification, cancellation refunds and live-order help actually work?
Price-parity audit	50 restaurants × 3 dishes	Does Ownly's price advantage hold across a real basket?

G. Data limitations

- No internal data: no orders, funnel, cancellations, refunds, tickets, dispatch, retention, CAC or unit economics.
- Customer reviews cover 8 days on Android only; one bad week could be over-represented.
- Keyword tagging produces false positives and misses regional-language reviews (11% of negatives untagged).
- Restaurant sample is small (33 problem reviews); captain sample skews to applicants.
- Semrush figures are modelled and desktop-only.
- Press facts were recorded in the source pack and not re-verified for this document.

H. Sources

- Google Play reviews: Ownly customer app (ownly_reviews.xlsx), Ownly partner app com.ownly.store (ownly_resturant.xlsx), Ownly Captain app (ownly_captain_reviews.xlsx). Scraped Oct 2026.
- Semrush: Organic Research Overview and Positions for ownly.food (India, desktop), 5 Oct 2026; Compare Domains and Organic Rankings screenshots.
- Press (secondary, via source pack OWNLY.docx): Business Today, 28 Jul 2026; Upstox, “Rapido's Ownly has early momentum”.
- All charts were computed directly from the review workbooks; scripts are available on request.